

Static analysis and software verification

Lecture 6 - Meaning of approximation

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it

Approximation

Intuition

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that
 - Soundness: $[[P]] \subseteq [[P]]^\#$

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that
 - Soundness: $[[P]] \subseteq [[P]]^\#$
 - Decidability: $[[P]]^\# \subseteq Q$

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that
 - Soundness: $[[P]] \subseteq [[P]]^\#$
 - Decidability: $[[P]]^\# \subseteq Q$
- Soundness is crucial since it allows us to exploit the decidability of approximation

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that
 - Soundness: $[[P]] \subseteq [[P]]^\#$
 - Decidability: $[[P]]^\# \subseteq Q$
- Soundness is crucial since it allows us to exploit the decidability of approximation
 - $[[P]]^\# \subseteq Q \implies [[P]] \subseteq Q$

Approximation

Intuition

- Let Q a property such that $[[P]] \subseteq Q$
 - In general, it is undecidable
- Thus, we compute an approximation $[[P]]^\#$ of $[[P]]$ such that
 - Soundness: $[[P]] \subseteq [[P]]^\#$
 - Decidability: $[[P]]^\# \subseteq Q$
- Soundness is crucial since it allows us to exploit the decidability of approximation
 - $[[P]]^\# \subseteq Q \implies [[P]] \subseteq Q$
 - Otherwise, we don't know

Abstraction

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)
- Can precisely describe some properties

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)
 - Can precisely describe some properties
 - Cannot describe all properties

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)
 - Can precisely describe some properties
 - Cannot describe all properties
- An abstraction in $\wp(\Sigma)$ of objects in Σ is $A \subseteq \wp(\Sigma)$ such that

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)
 - Can precisely describe some properties
 - Cannot describe all properties
- An abstraction in $\wp(\Sigma)$ of objects in Σ is $A \subseteq \wp(\Sigma)$ such that
 - The elements in A are those precisely described by the abstraction (without loss of precision)

Abstraction

- Abstraction is the process of replacing something *concrete* by some description dealing with *some* properties (in general not all)
 - Can precisely describe some properties
 - Cannot describe all properties
- An abstraction in $\wp(\Sigma)$ of objects in Σ is $A \subseteq \wp(\Sigma)$ such that
 - The elements in A are those precisely described by the abstraction (without loss of precision)
 - The elements not in A have to be represented by other elements of A (with loss of precision)

Objects

Objects

- When analyzing programs we have to consider object representing parts of the computation state
 - Values: booleans, $\mathbb{Z}$, etc
 - Variables
 - Environments
 - Stacks
 - ...

Properties

Properties

- Properties are set of objects (having that property)
 - Odd naturals: $\{1, 3, 5, \dots, 2n + 1, \dots\}$
 - Even integers: $\{2z \mid z \in \mathbb{Z}\}$
 - ...

The lattice of properties

The lattice of properties

- The set of properties $\wp(\Sigma)$ of objects Σ is a complete distributive lattice

The lattice of properties

- The set of properties $\wp(\Sigma)$ of objects Σ is a complete distributive lattice

$$\langle \wp(\Sigma), \subseteq, \emptyset, \cup, \cap, \neg \rangle$$

The lattice of properties

- The set of properties $\wp(\Sigma)$ of objects Σ is a complete distributive lattice

$$\langle \wp(\Sigma), \subseteq, \emptyset, \cup, \cap, \neg \rangle$$

- A property is a subset of Σ
- $\subseteq$ corresponds to the logical implication between properties
- Σ is true, $\emptyset$ is false
- $\cup$ is disjunction, $\cap$ is conjunction, $\neg$ is negation

Direction of abstraction

Direction of abstraction

- When approximating a concrete property P with an abstraction one $P^\sharp$, we may consider two cases

Direction of abstraction

- When approximating a concrete property P with an abstraction one $P^\sharp$, we may consider two cases
 - Approximation from below: $P^\sharp \subseteq P$

Direction of abstraction

- When approximating a concrete property P with an abstraction one $P^\sharp$, we may consider two cases
 - Approximation from below: $P^\sharp \subseteq P$
 - Approximation from above: $P \subseteq P^\sharp$

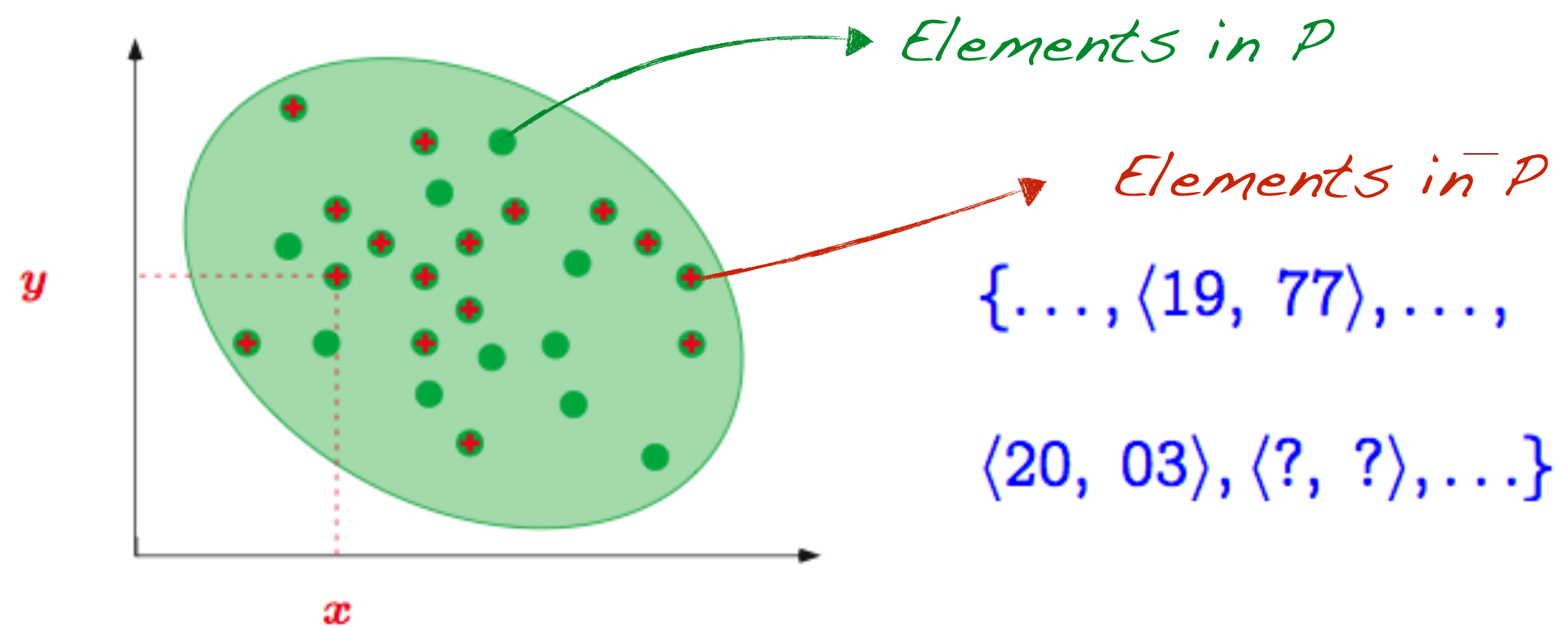
Direction of abstraction

- When approximating a concrete property P with an abstraction one $P^\sharp$, we may consider two cases
 - Approximation from below: $P^\sharp \subseteq P$
 - Approximation from above: $P \subseteq P^\sharp$
 - Other relation could be considered

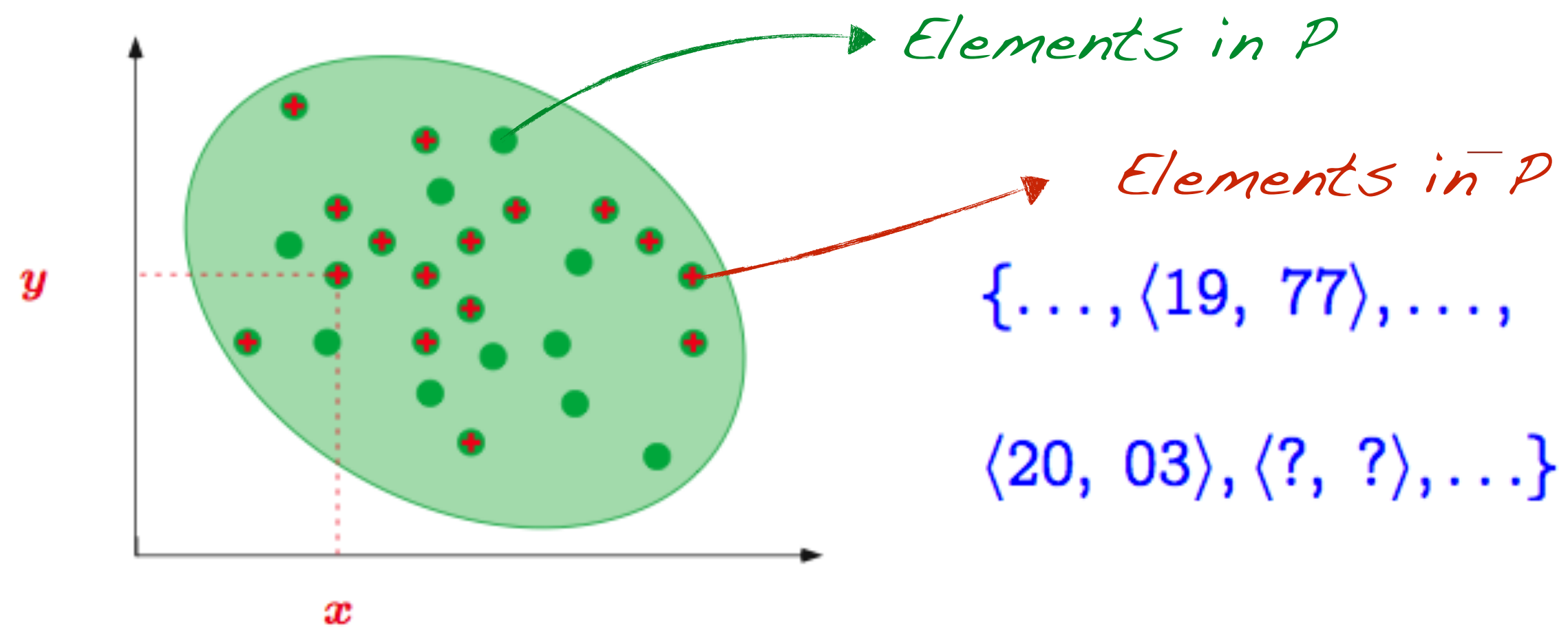
Direction of abstraction

- When approximating a concrete property P with an abstraction one $P^\sharp$, we may consider two cases
 - Approximation from below: $P^\sharp \subseteq P$
 - Approximation from above: $P \subseteq P^\sharp$
 - Other relation could be considered
- The two notion are dual (we will focus on the second one)

Approximation from below

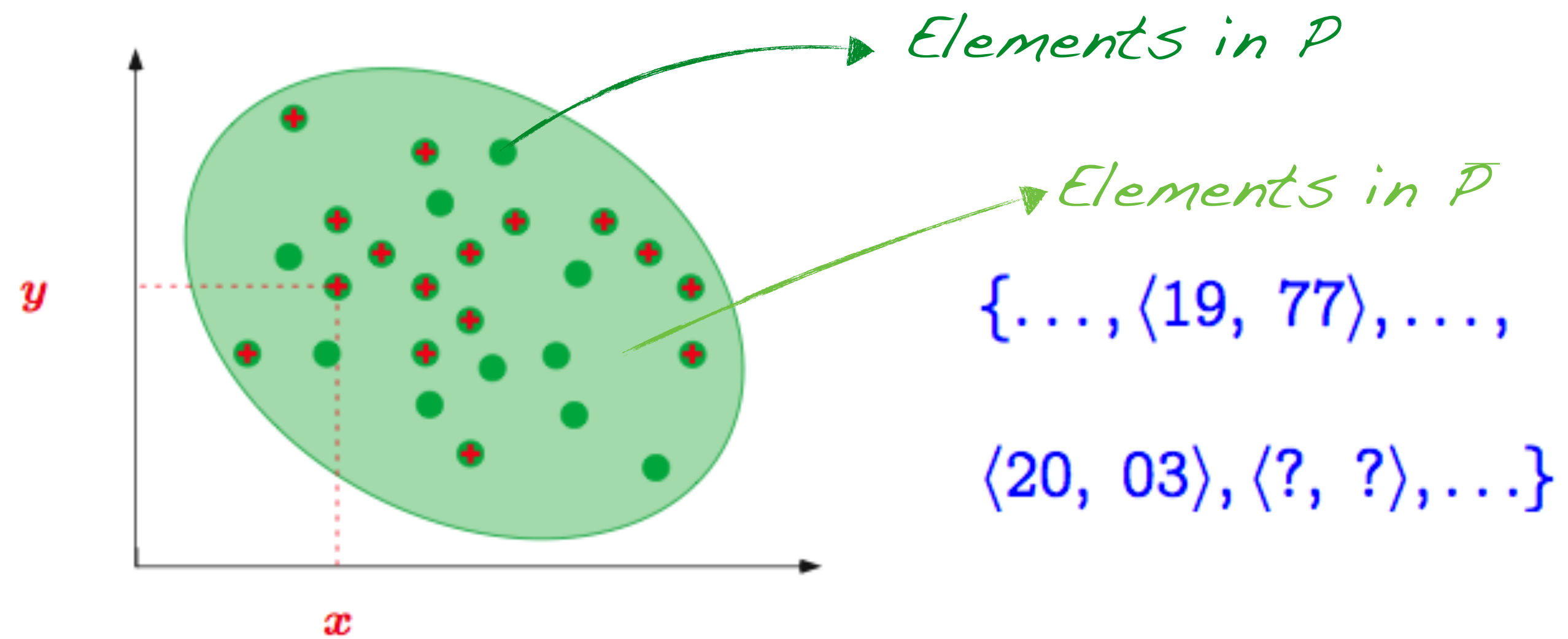


Approximation from below

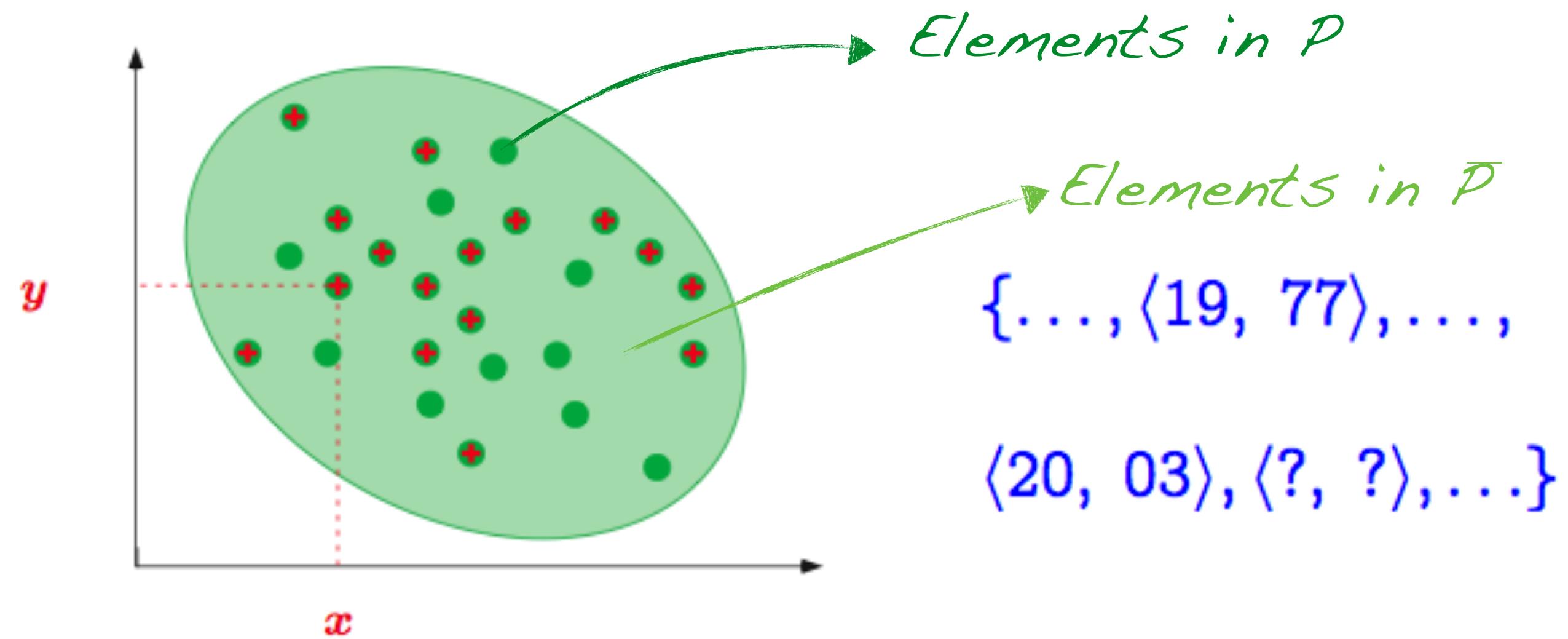


- To answer $\langle x, y \rangle \in P$ using only $P^\#$, such that $P^\# \subseteq P$
 - if $\langle x, y \rangle \notin P^\#$ then “I don’t know”
 - if $\langle x, y \rangle \in P^\#$ then yes

Approximation from above

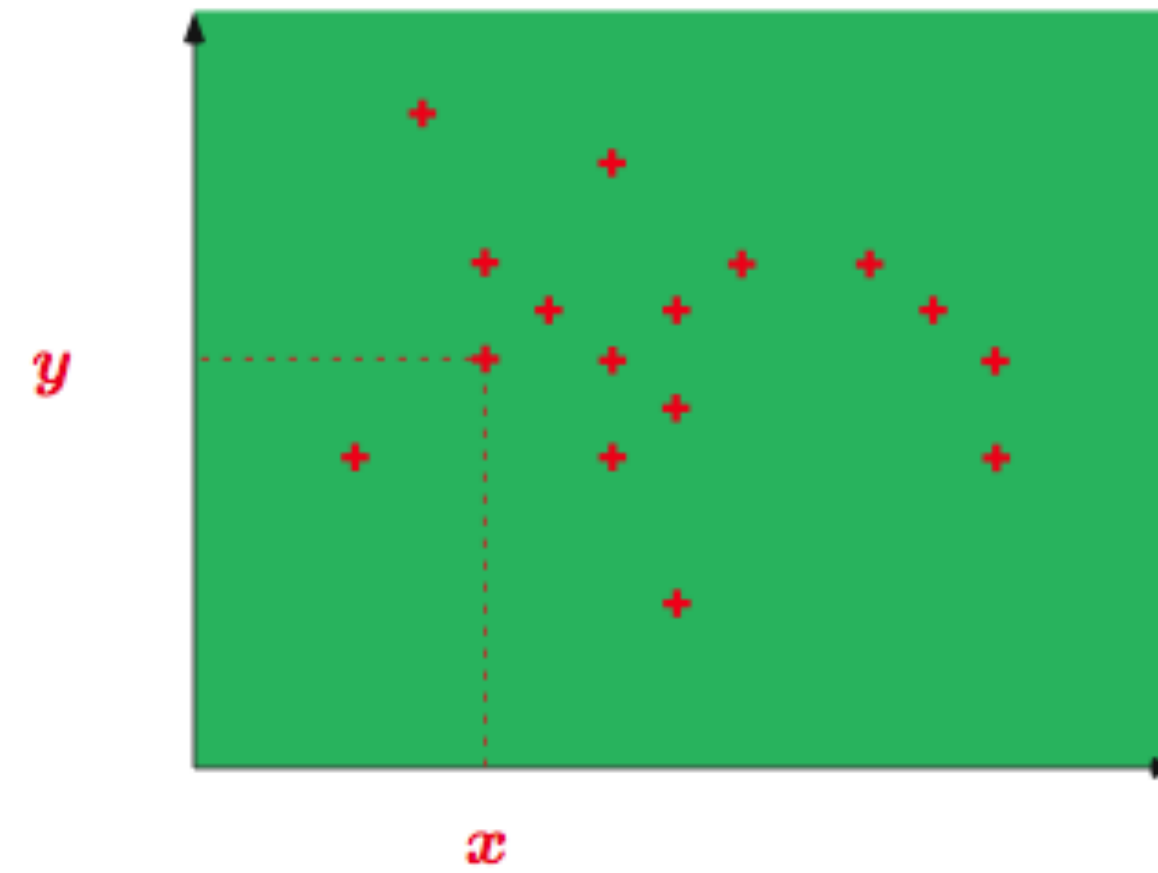


Approximation from above



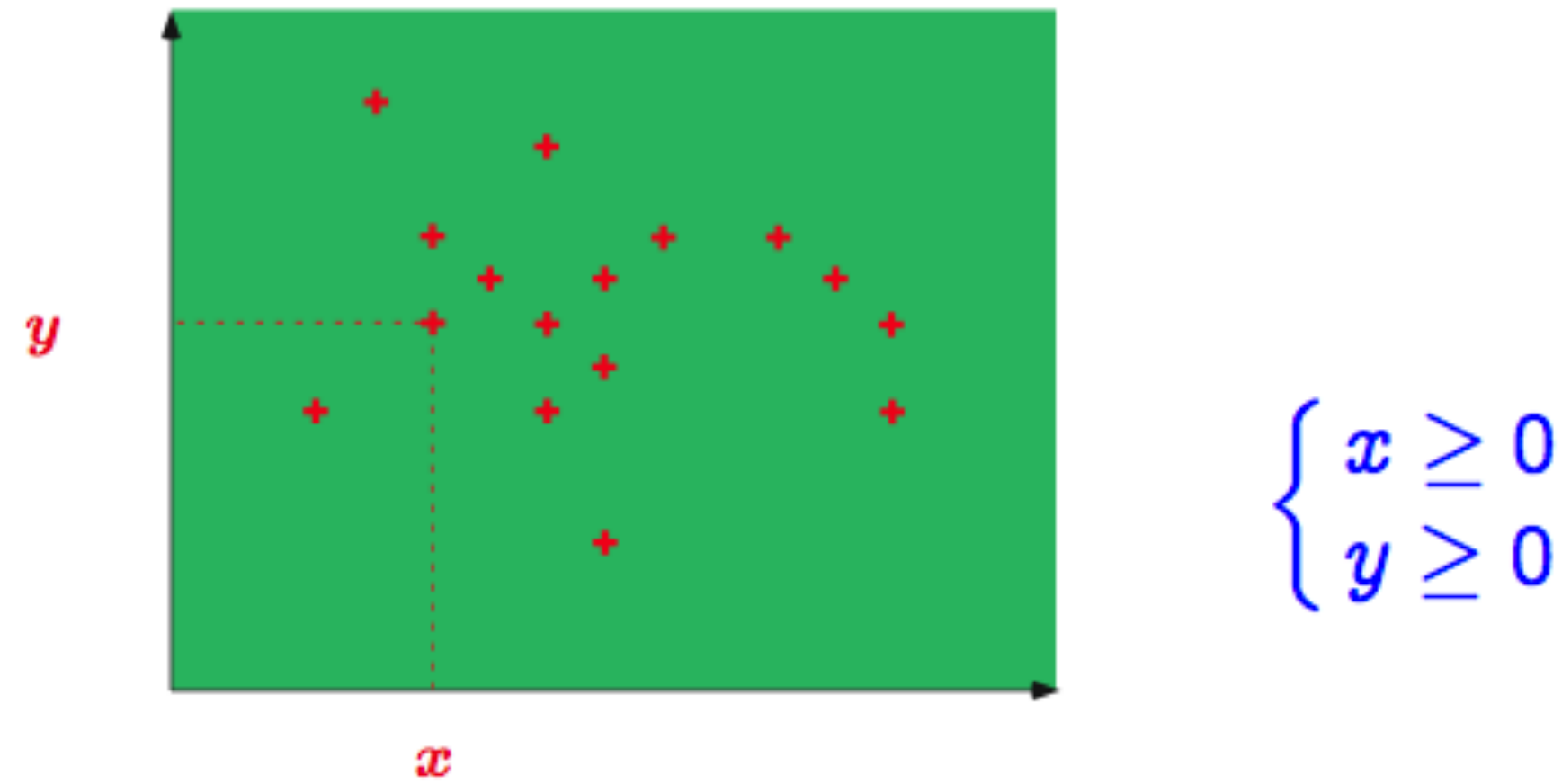
- To answer $\langle x, y \rangle \in P$ using only $P^\#$, such that $P \subseteq P^\#$
 - if $\langle x, y \rangle \notin P^\#$ then no
 - if $\langle x, y \rangle \in P^\#$ then “I don’t know”

An example: Sign



$$\begin{cases} x \geq 0 \\ y \geq 0 \end{cases}$$

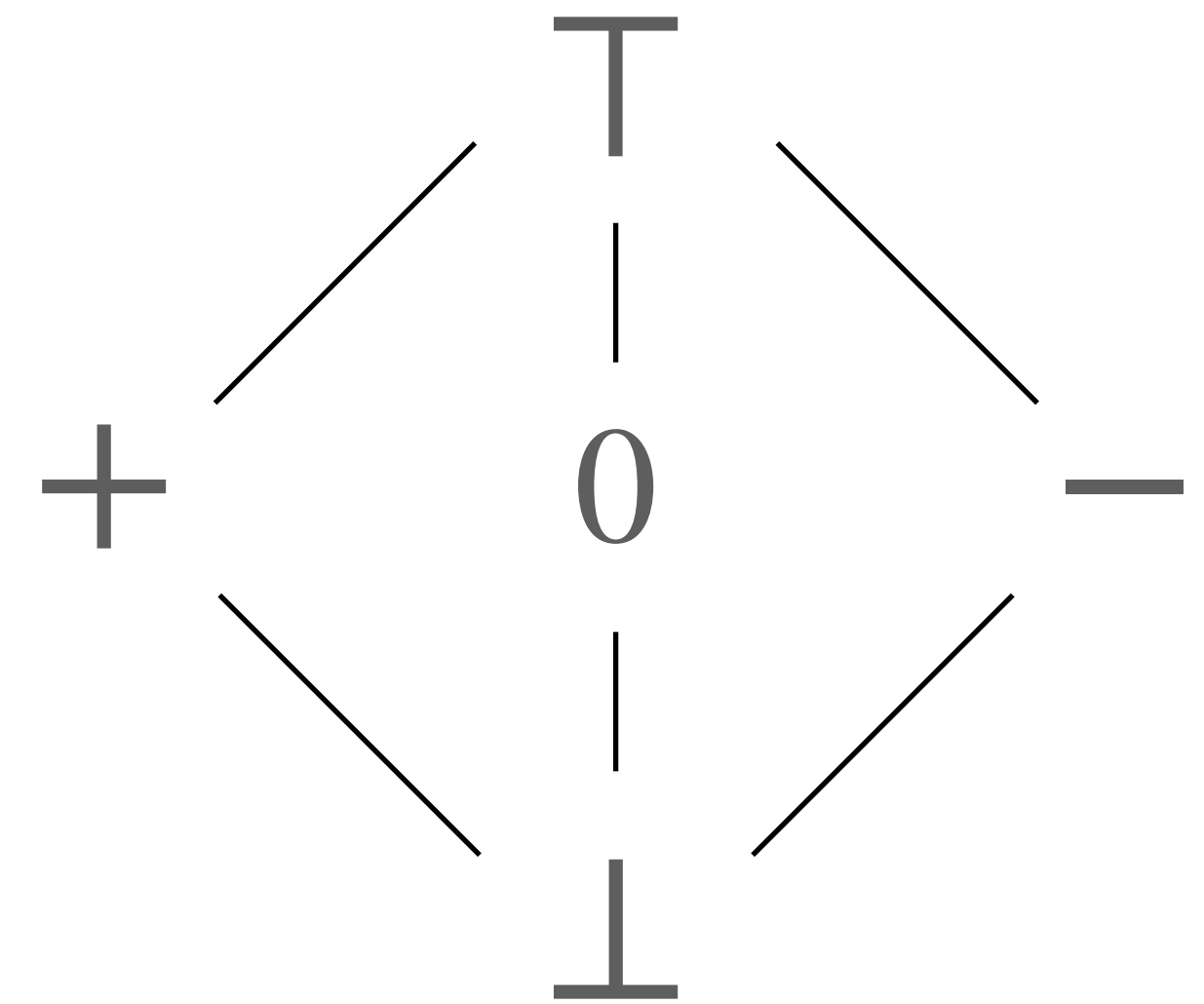
An example: Sign



- Programs: integer manipulation programs
- Properties: the sign of integer variables
- Concrete semantics: standard operations manipulating integers
- Abstract semantics: abstract operations (= sign rules)

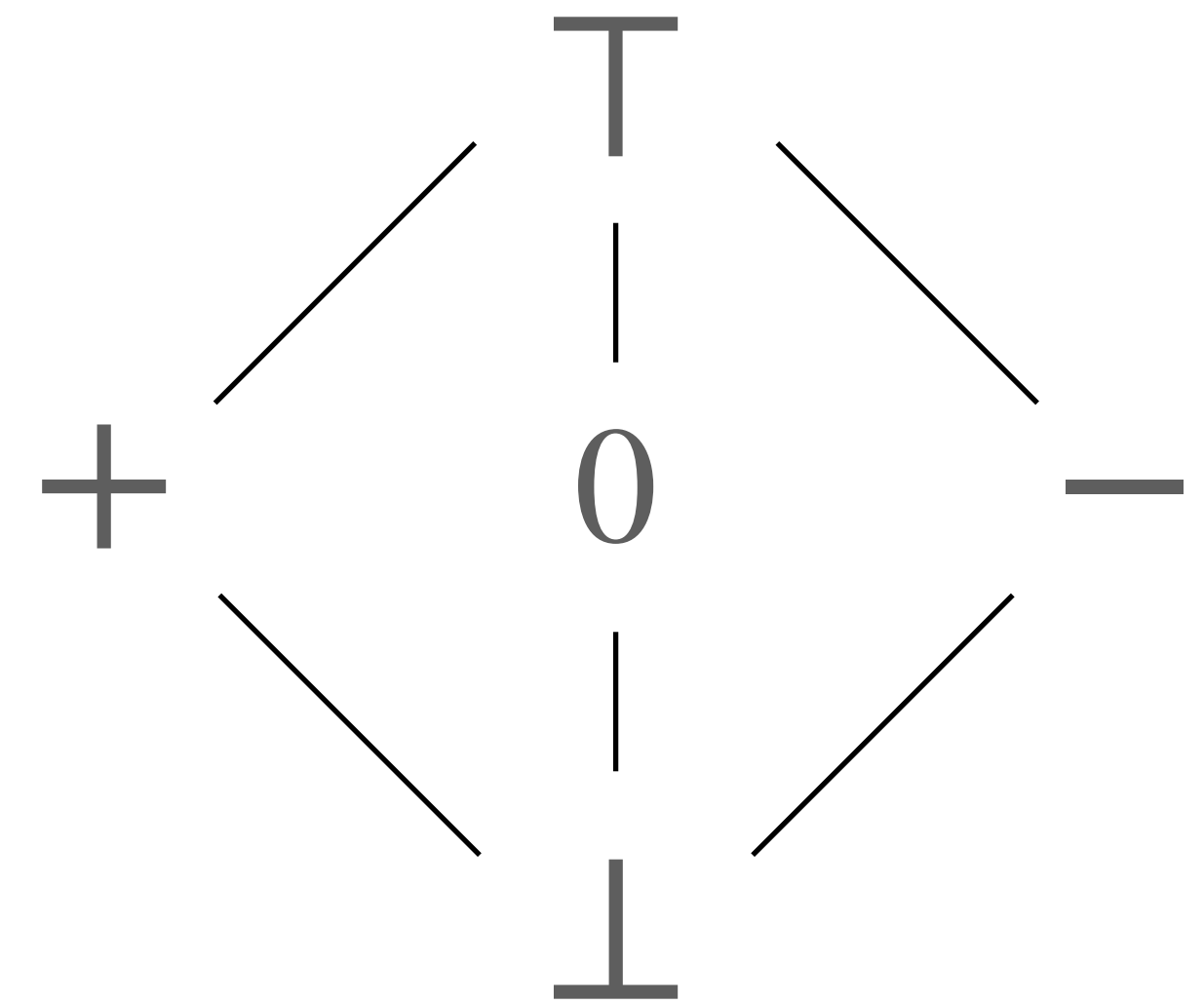
Objects' abstraction

- Concrete domain: $\wp(\mathbb{Z})$
- The property we want to study is the sign of variables
 - + represents positive integers without 0
 - - represents negative integers without 0
 - 0 represents the singleton set $\{0\}$



Objects' abstraction

- The sign domain is contained in $\wp(\mathbb{Z})$
- A set X in $\wp(\mathbb{Z})$ is approximated by the smallest set of integers which contains X and it is represented in the sign domain



Approximating operators

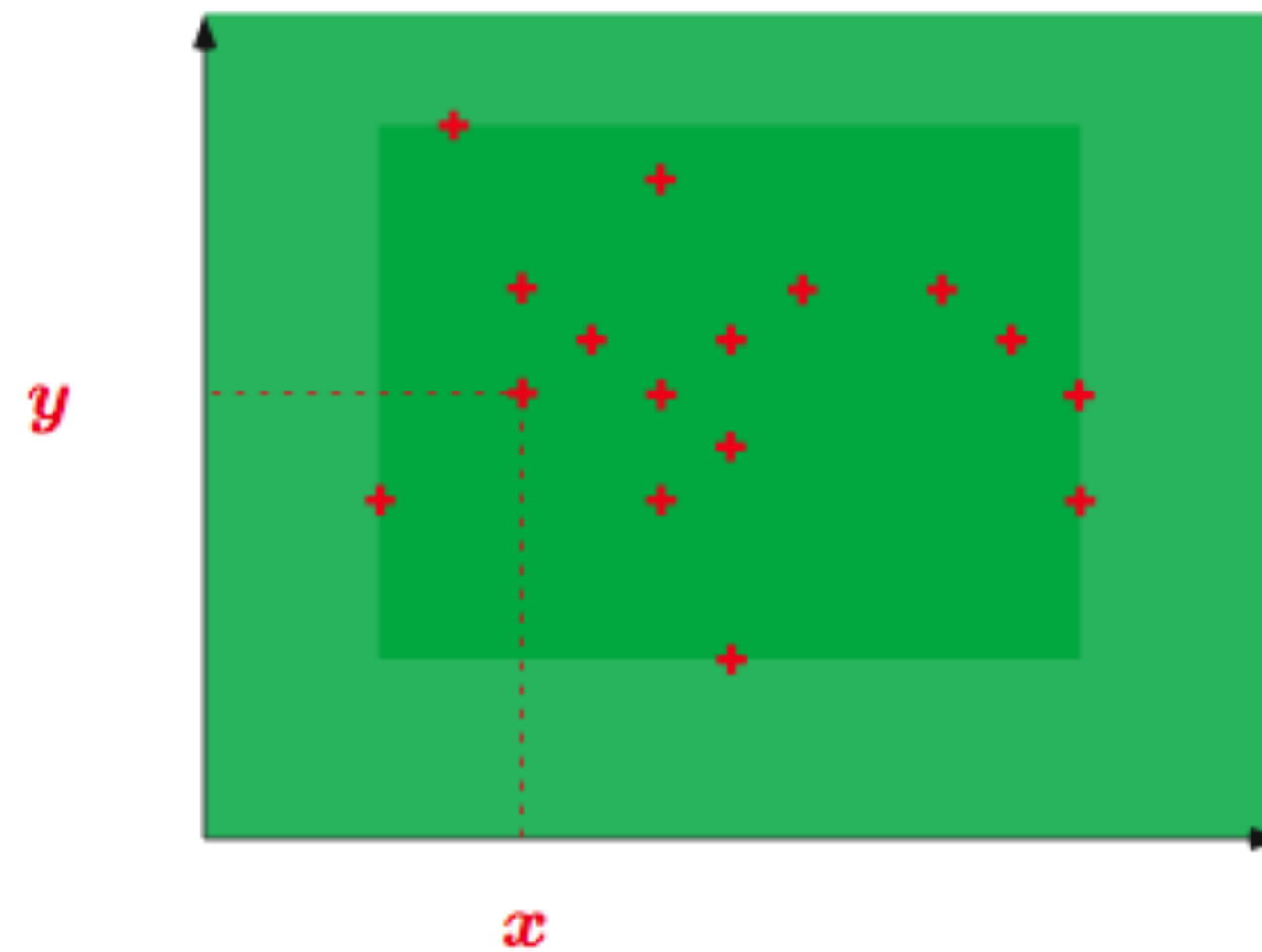
$\oplus$	$\mathbb{Z}$	$+$	$-$	0	$\emptyset$
$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$
$+$	$\mathbb{Z}$	$+$	$\mathbb{Z}$	$+$	$+$
$-$	$\mathbb{Z}$	$\mathbb{Z}$	$-$	$-$	$-$
0	$\mathbb{Z}$	$+$	$-$	0	0
$\emptyset$	$\mathbb{Z}$	$+$	$-$	0	$\emptyset$

$\otimes$	$\mathbb{Z}$	$+$	$-$	0	$\emptyset$
$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	0	$\mathbb{Z}$
$+$	$\mathbb{Z}$	$+$	$-$	0	$+$
$-$	$\mathbb{Z}$	$-$	$-$	0	$-$
0	0	0	0	0	0
$\emptyset$	$\mathbb{Z}$	$+$	$-$	0	$\emptyset$

wrong

Other examples

Intervals⁵

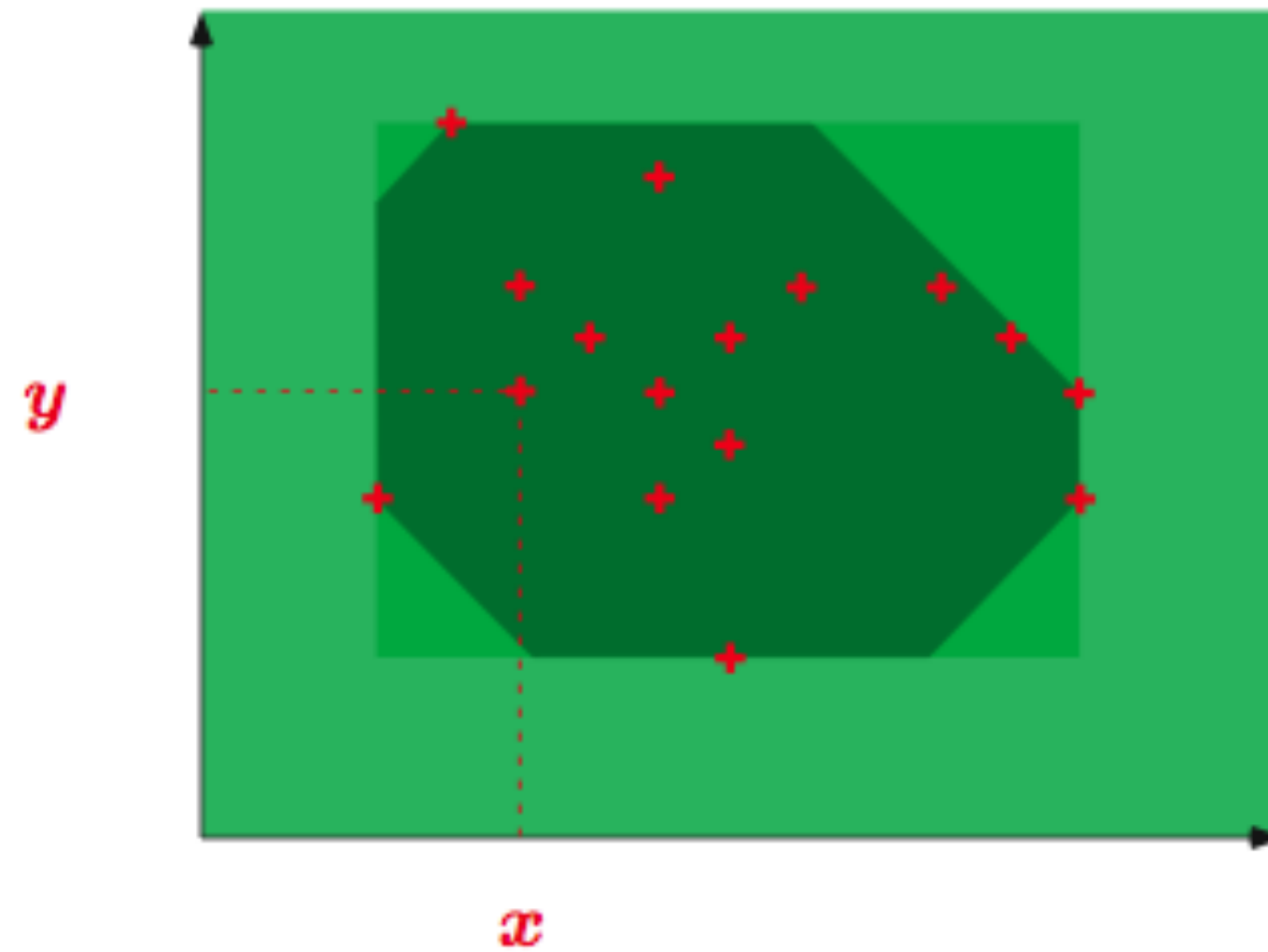


$$\begin{cases} x \in [19, 77] \\ y \in [20, 03] \end{cases}$$

⁵ P. Cousot & R. Cousot. *Static determination of dynamic properties of programs*. Proc. 2nd Int. Symp. on Programming, Dunod, 1976.

Other examples

Octagons⁶

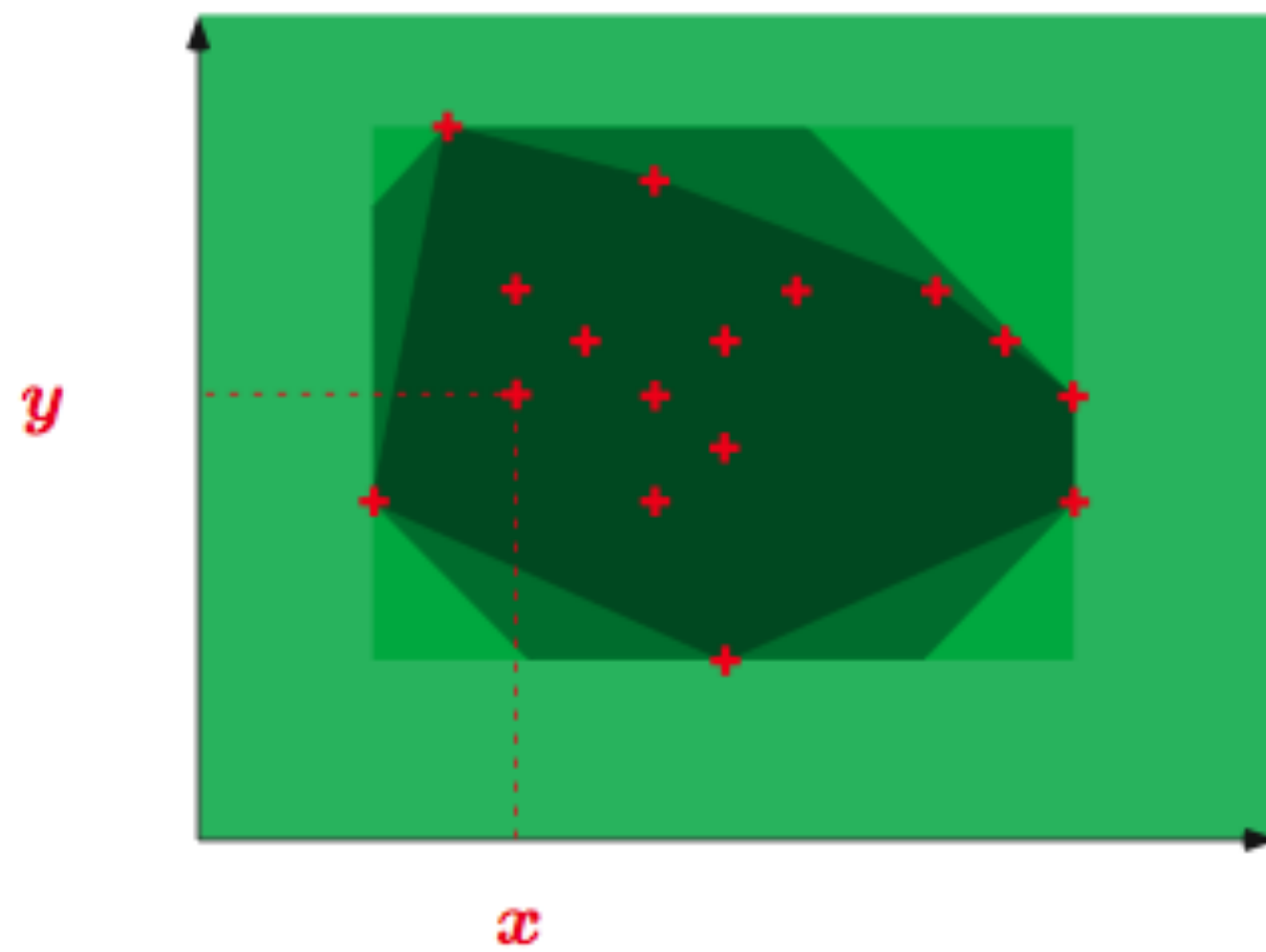


$$\begin{cases} 1 \leq x \leq 9 \\ x + y \leq 77 \\ 1 \leq y \leq 9 \\ x - y \leq 99 \end{cases}$$

⁶ A. Miné. *A New Numerical Abstract Domain Based on Difference-Bound Matrices*. PADO'2001 LNCS 2053, pp. 155–172. Springer 2001. See the *The Octagon Abstract Domain Library* on <http://www.di.ens.fr/~mine/oct/>

Other examples

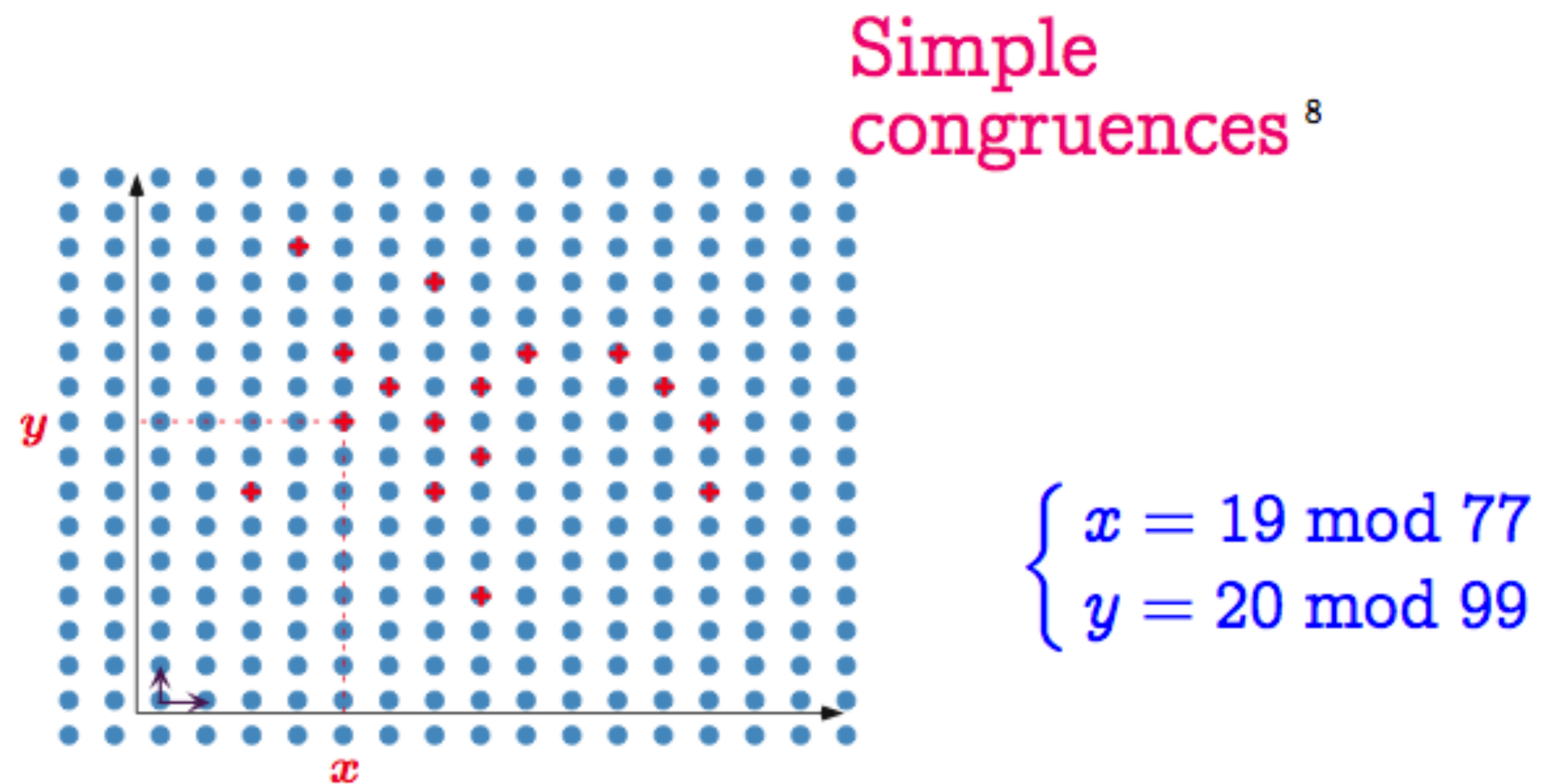
Polyhedra⁷



$$\begin{cases} 19x + 77y \leq 2004 \\ 20x + 03y \geq 0 \end{cases}$$

⁷ P. Cousot & N. Halbwachs. *Automatic discovery of linear restraints among variables of a program*. ACM POPL, 1978, pp. 84–97.

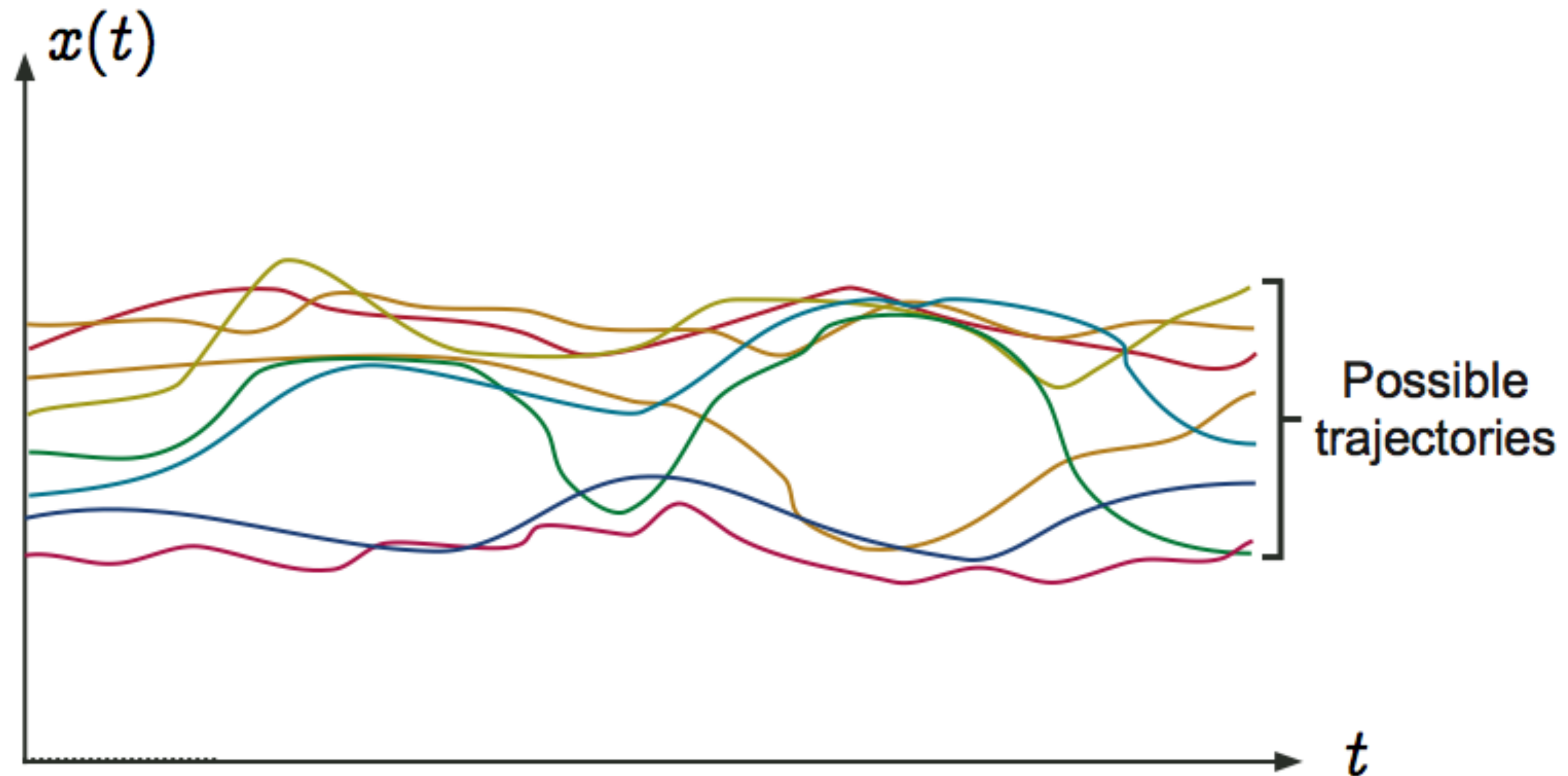
Other examples



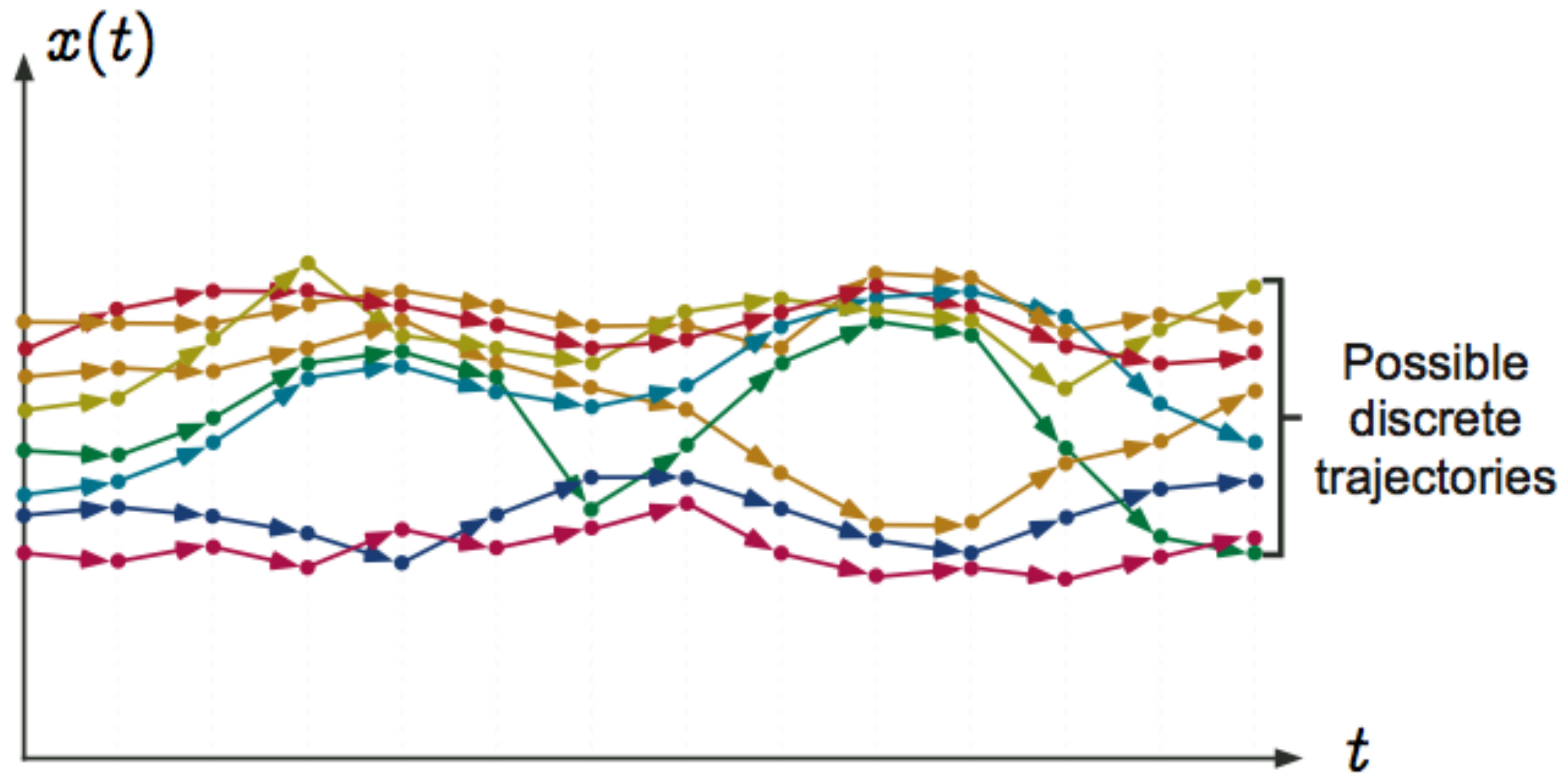
⁸ Ph. Granger. *Static Analysis of Arithmetical Congruences*. Int. J. Comput. Math. 30, 1989, pp. 165–190.

Approximating computations

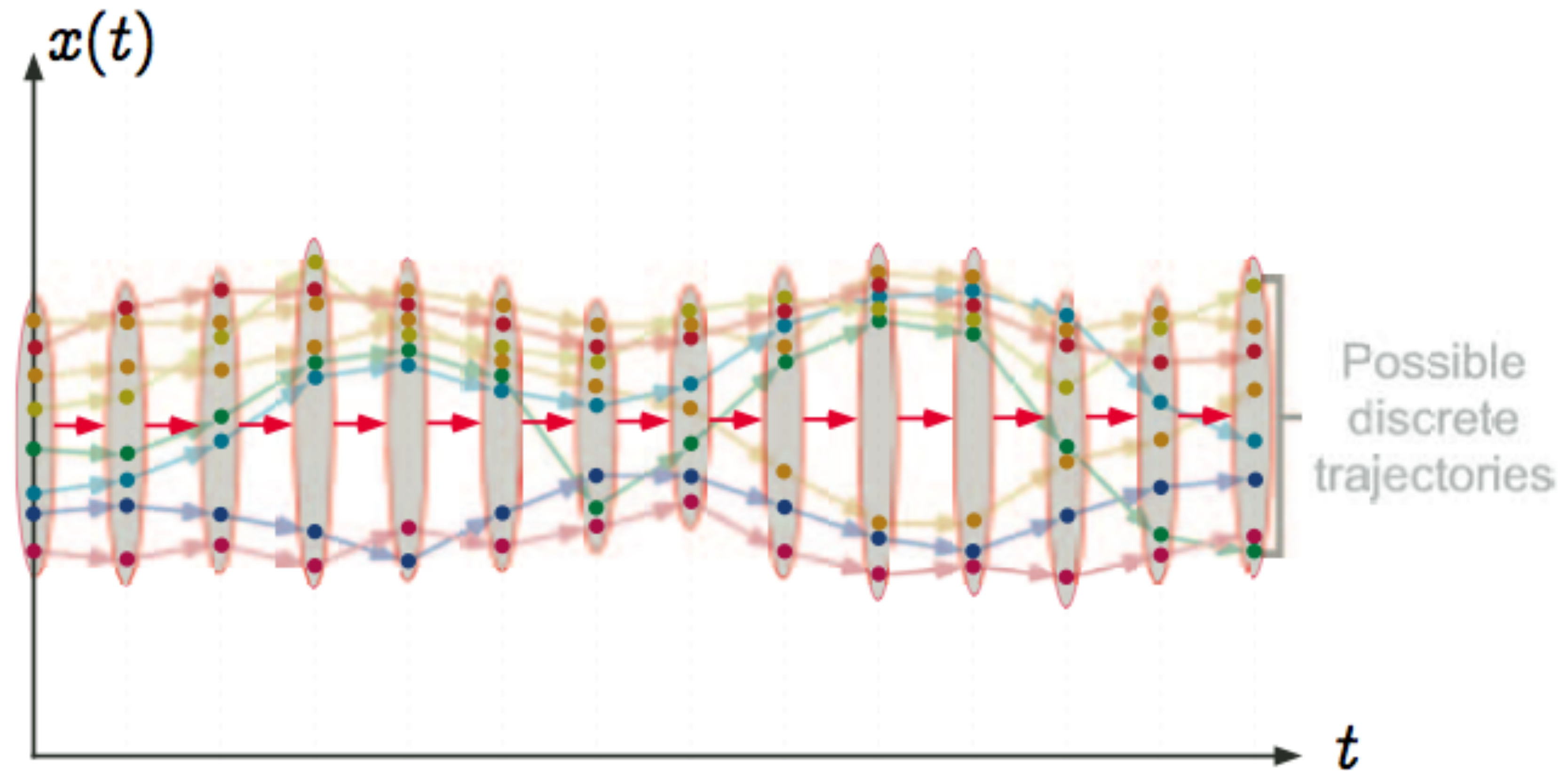
Possible behaviors



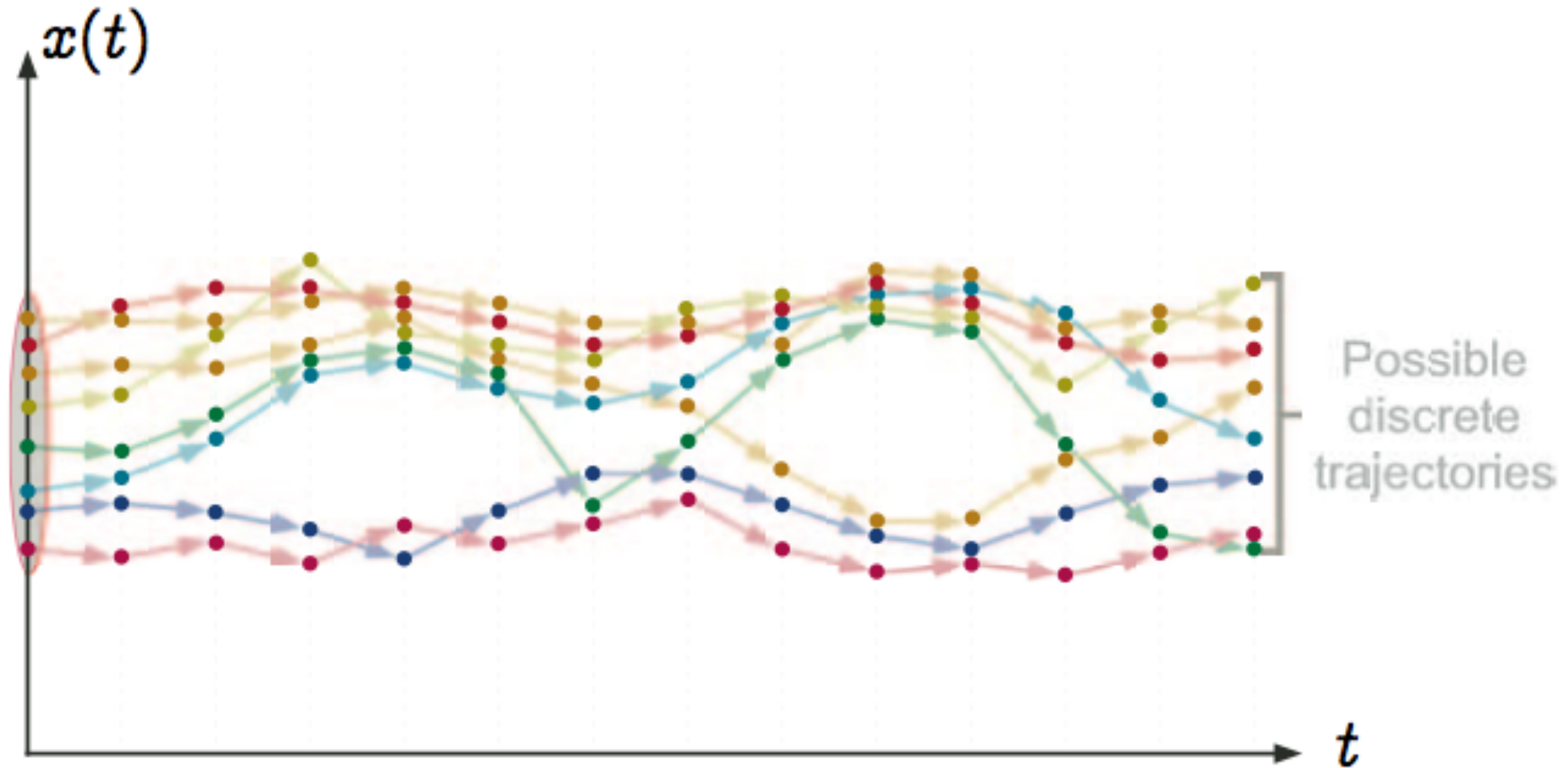
Small-step transition semantics



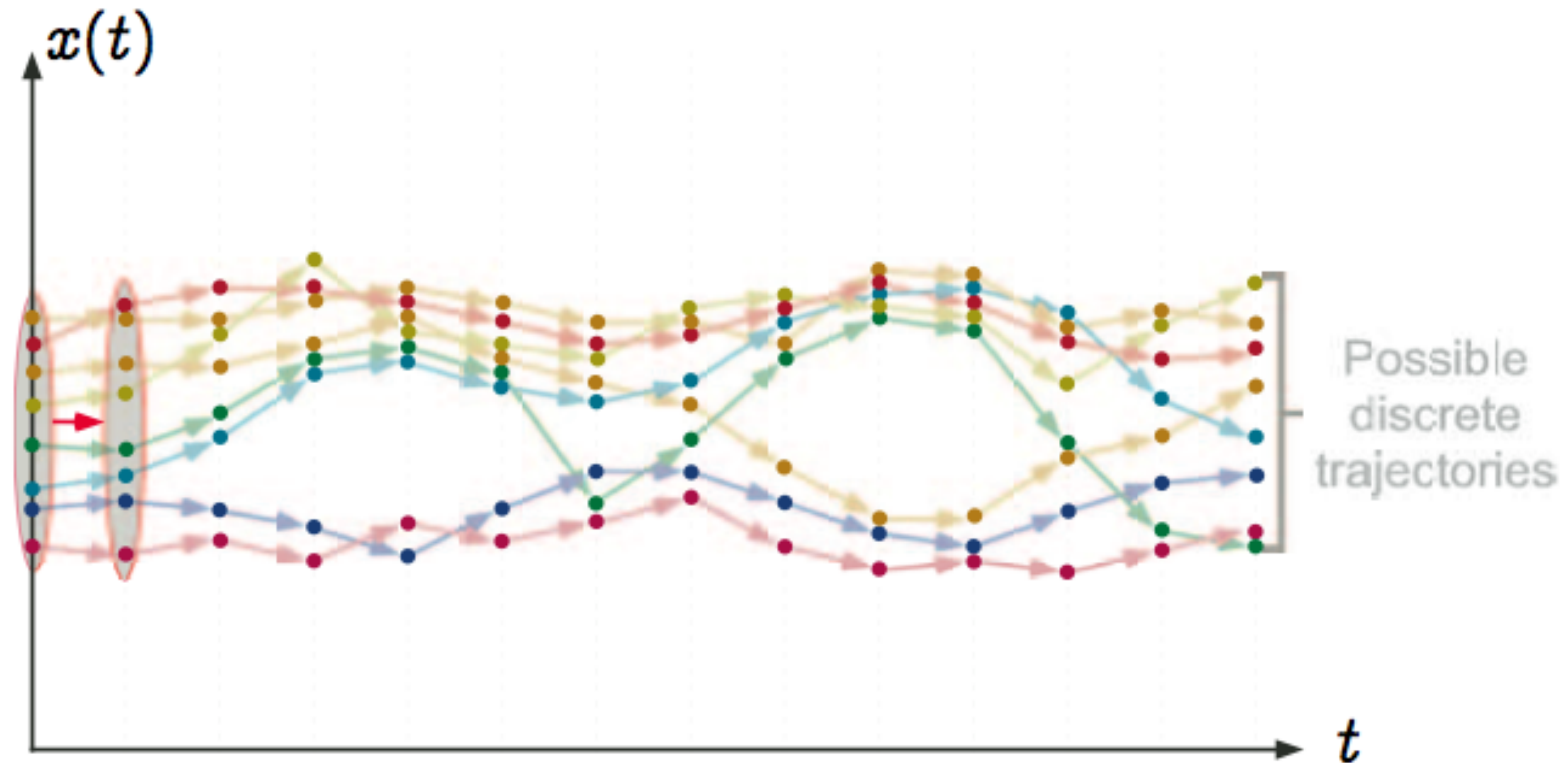
Computing on sets



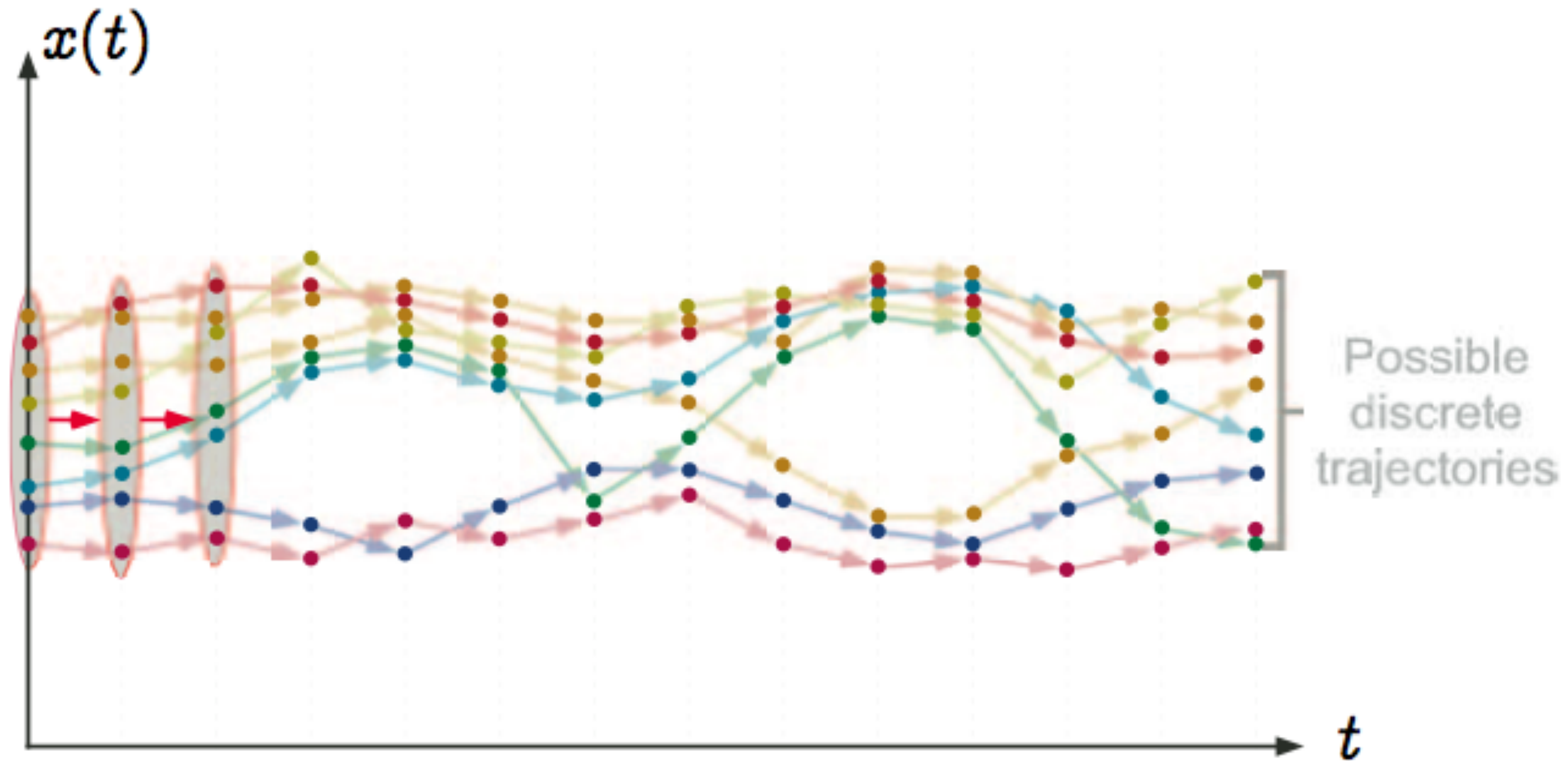
Computing on set by fixpoint



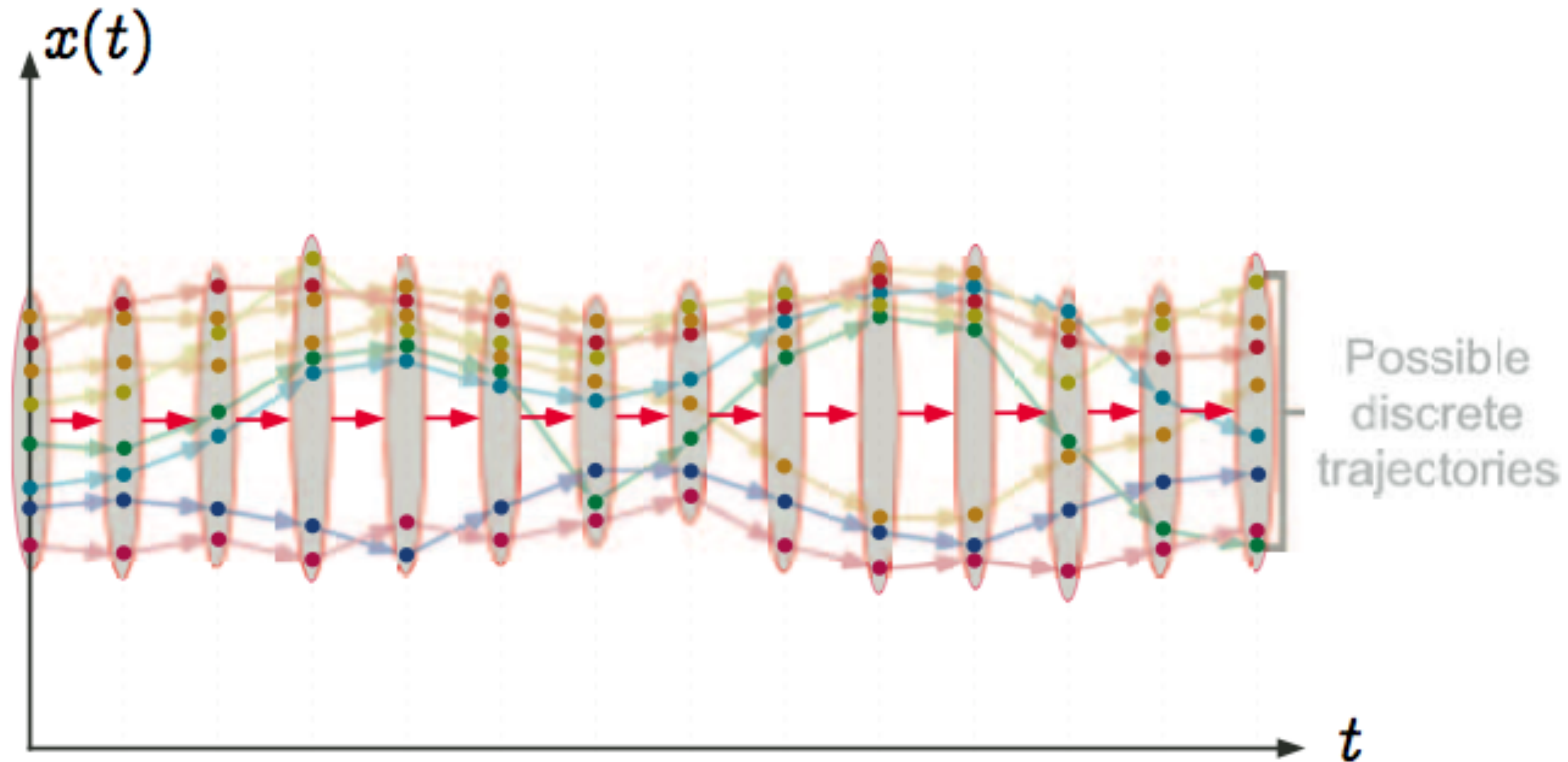
Computing on set by fixpoint



Computing on set by fixpoint

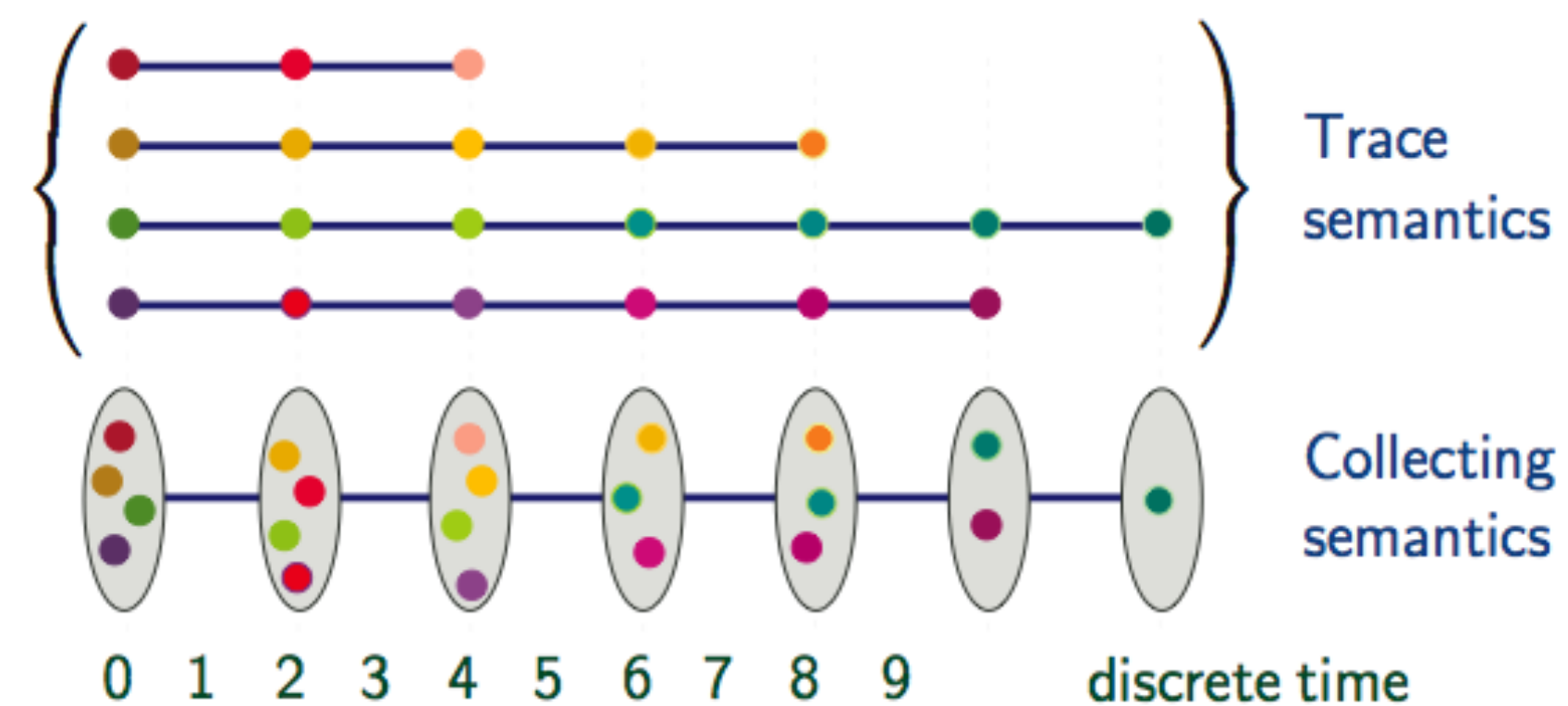


Computing on set by fixpoint



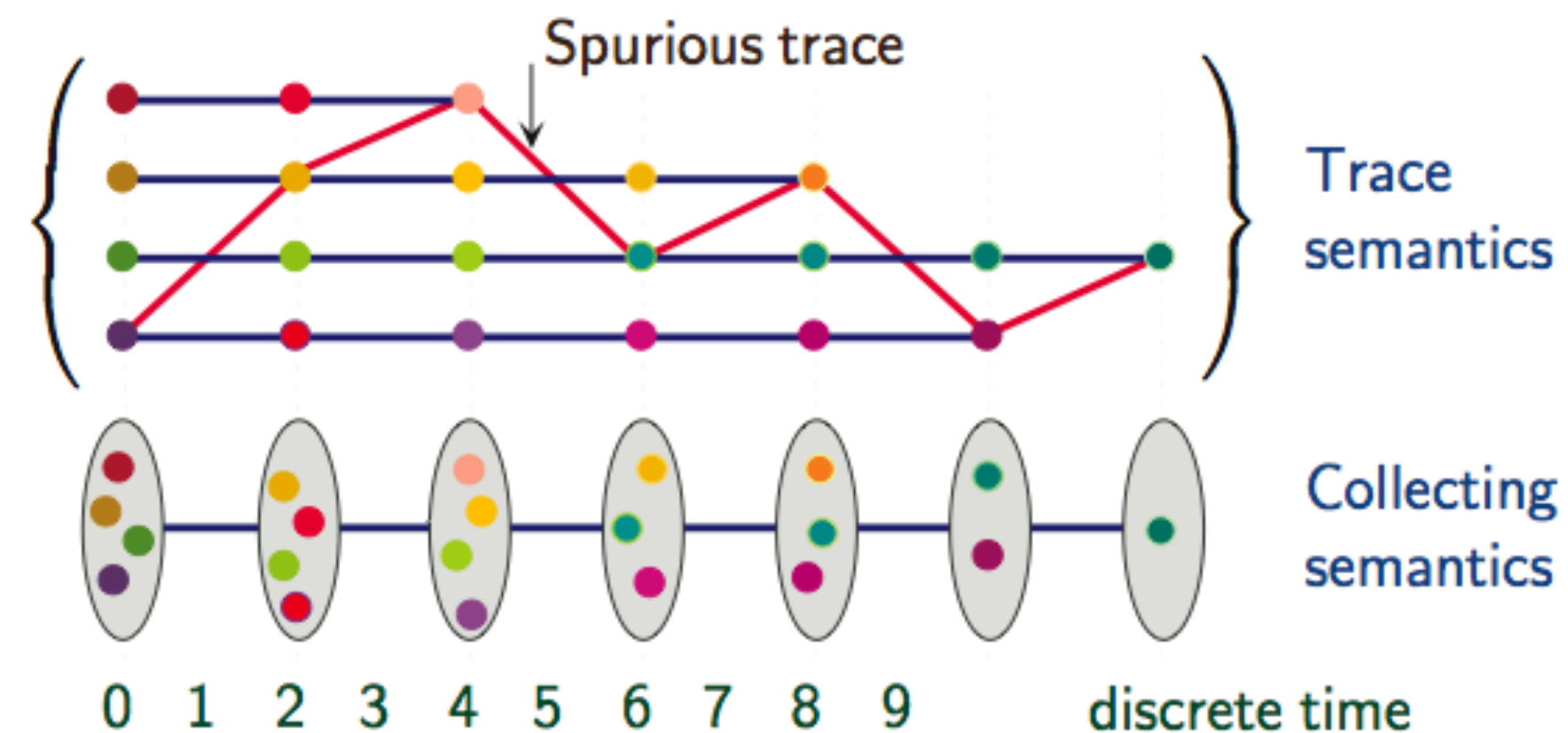
Where is the approximation?

- Collect all states that can appear on some trace at any discrete time (this is already an abstraction!)



Where is the approximation?

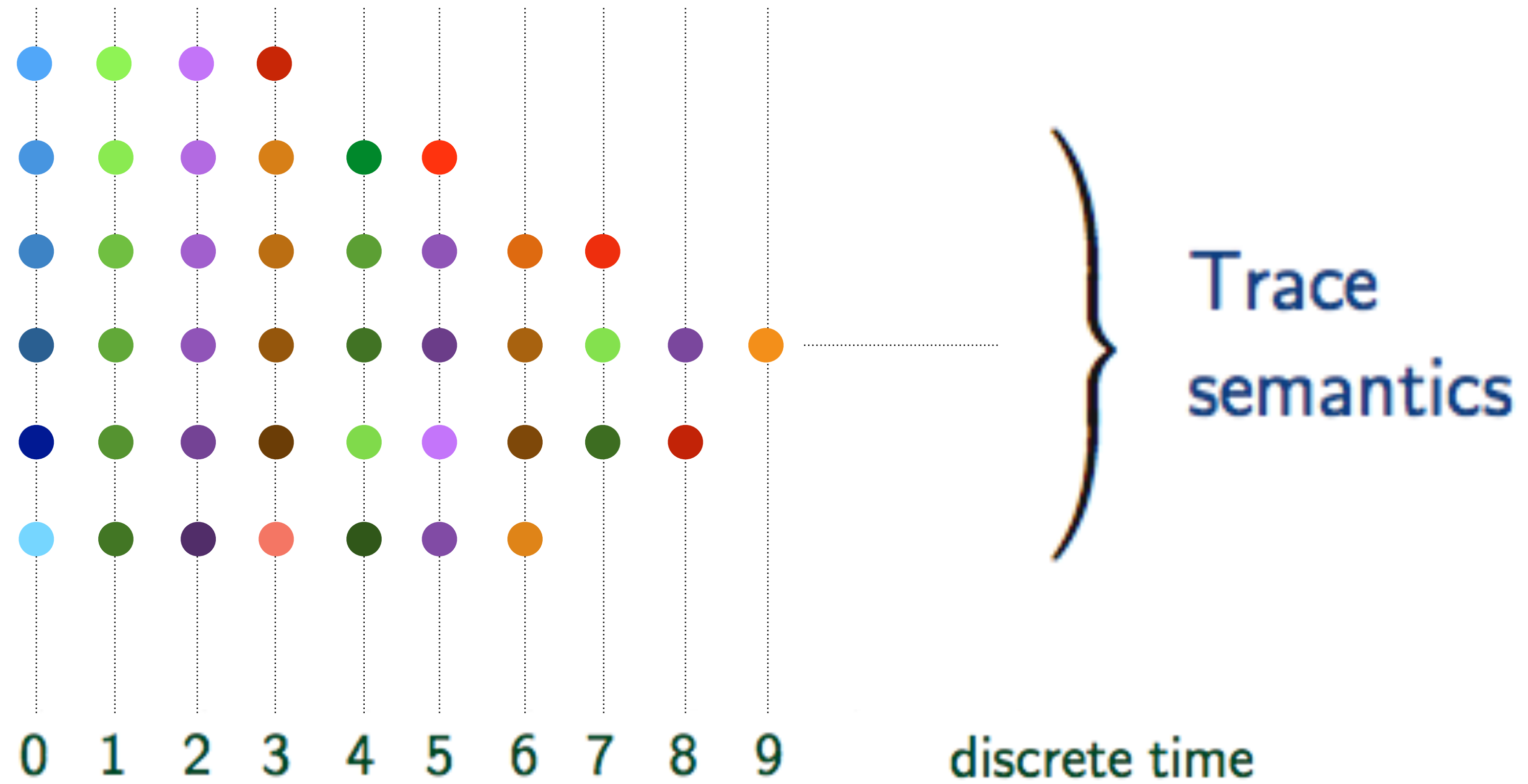
- Collect all states that can appear on some trace at any discrete time (this is already an abstraction!)



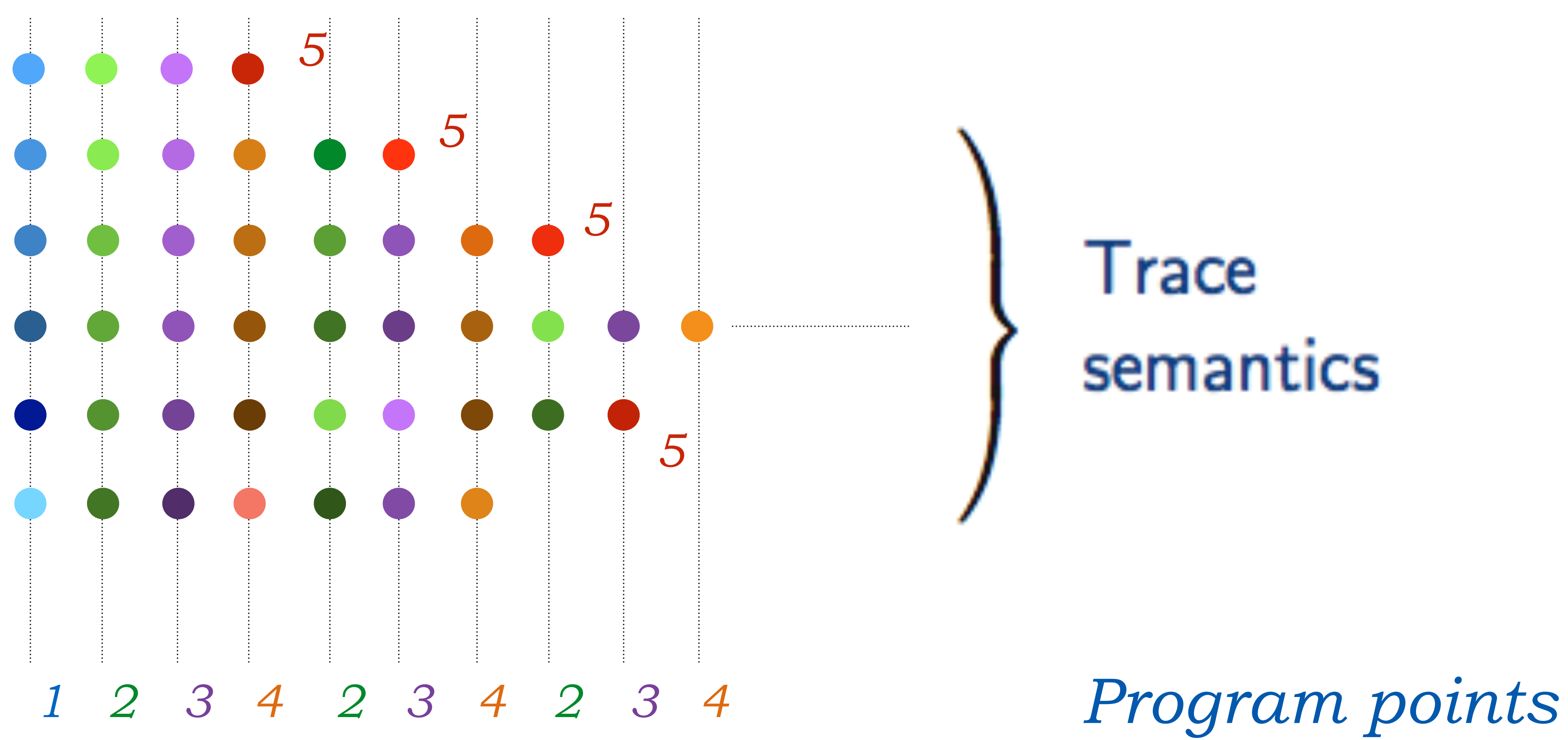
What still we need

- Tracing collections of states instead of single states: we have spurious execution due to collection (but only among reachable states)
- We have to further collect computations, computing on program points instead of on steps of computations

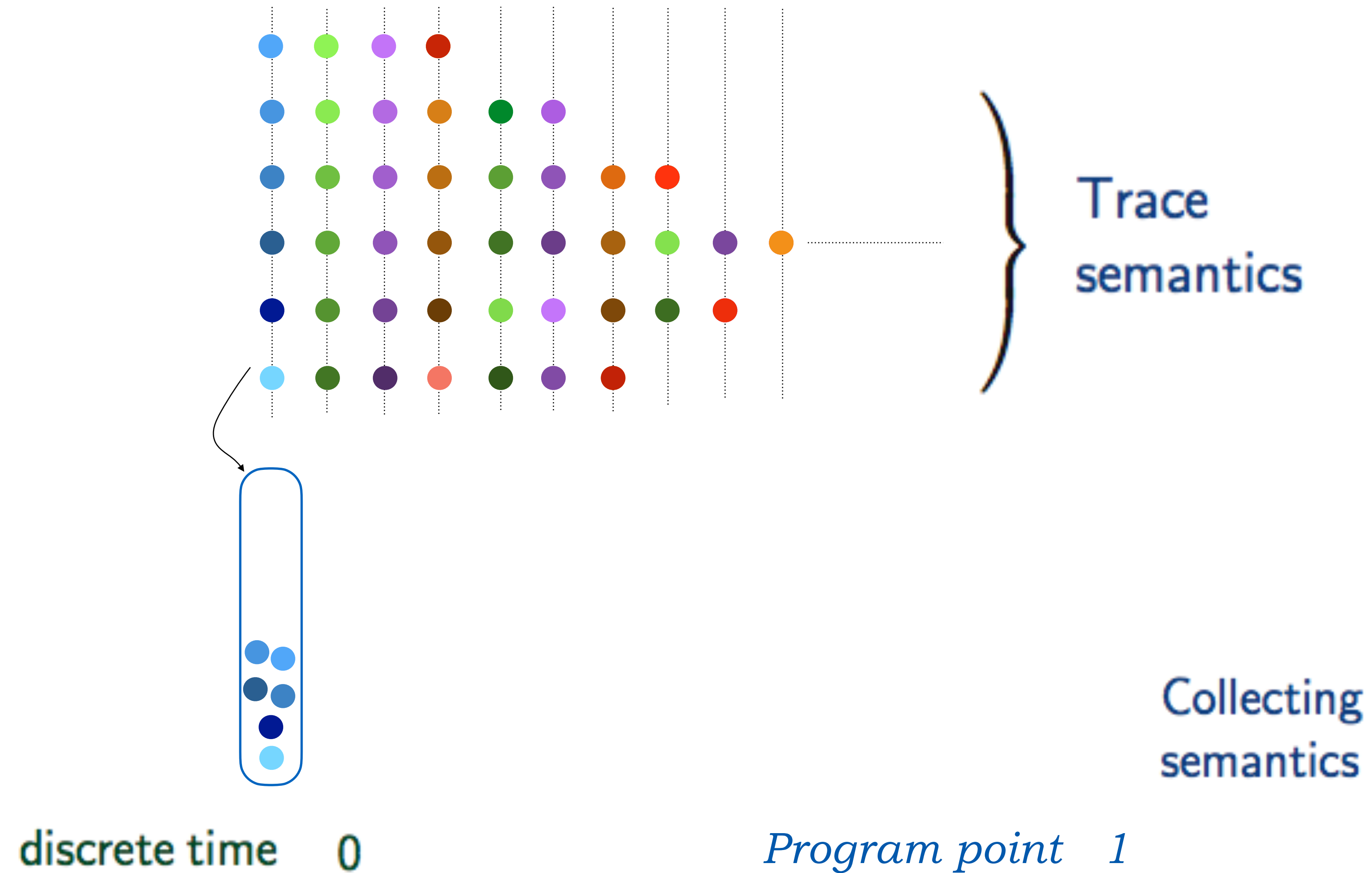
Collecting semantics



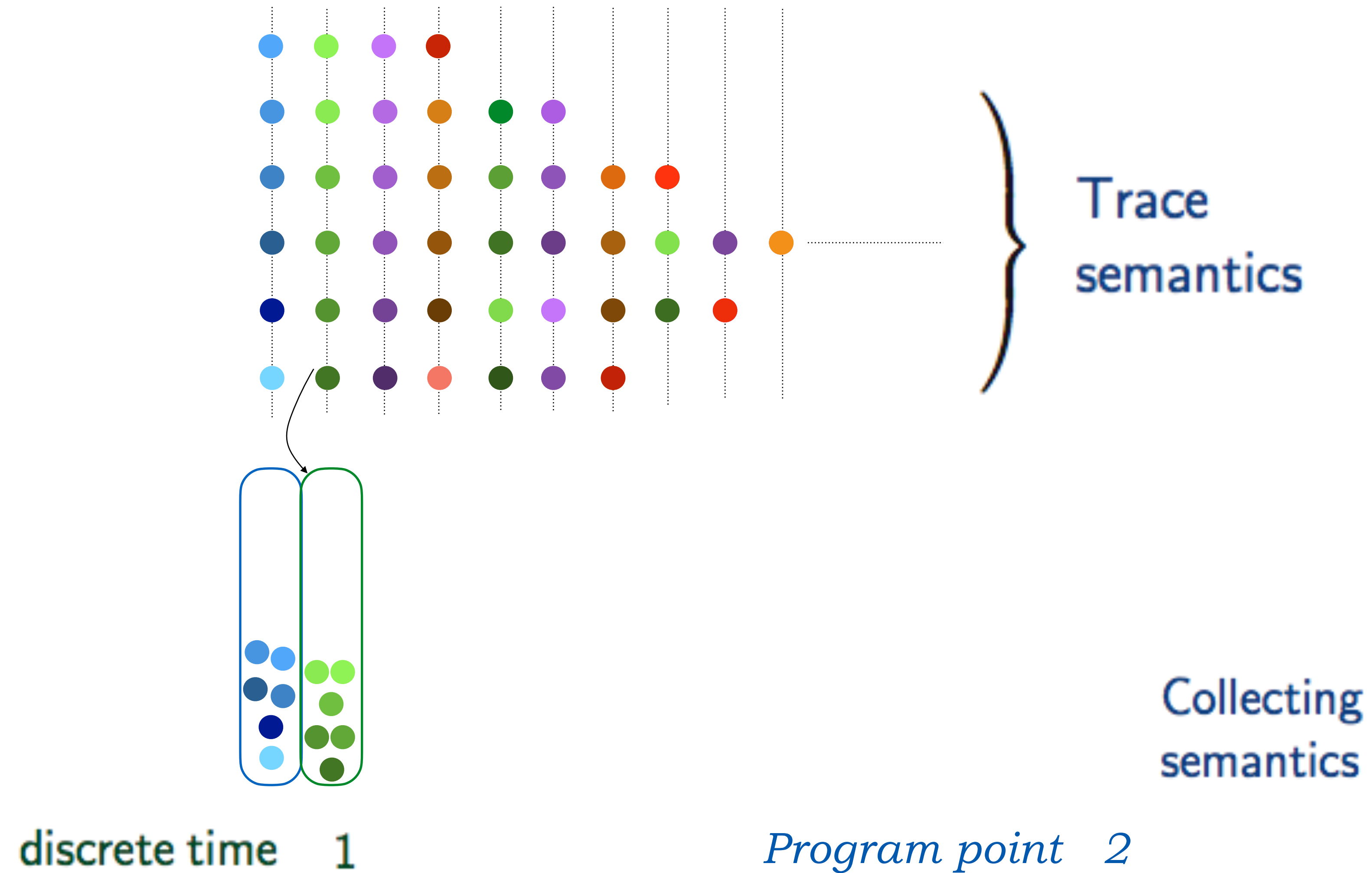
Collecting semantics



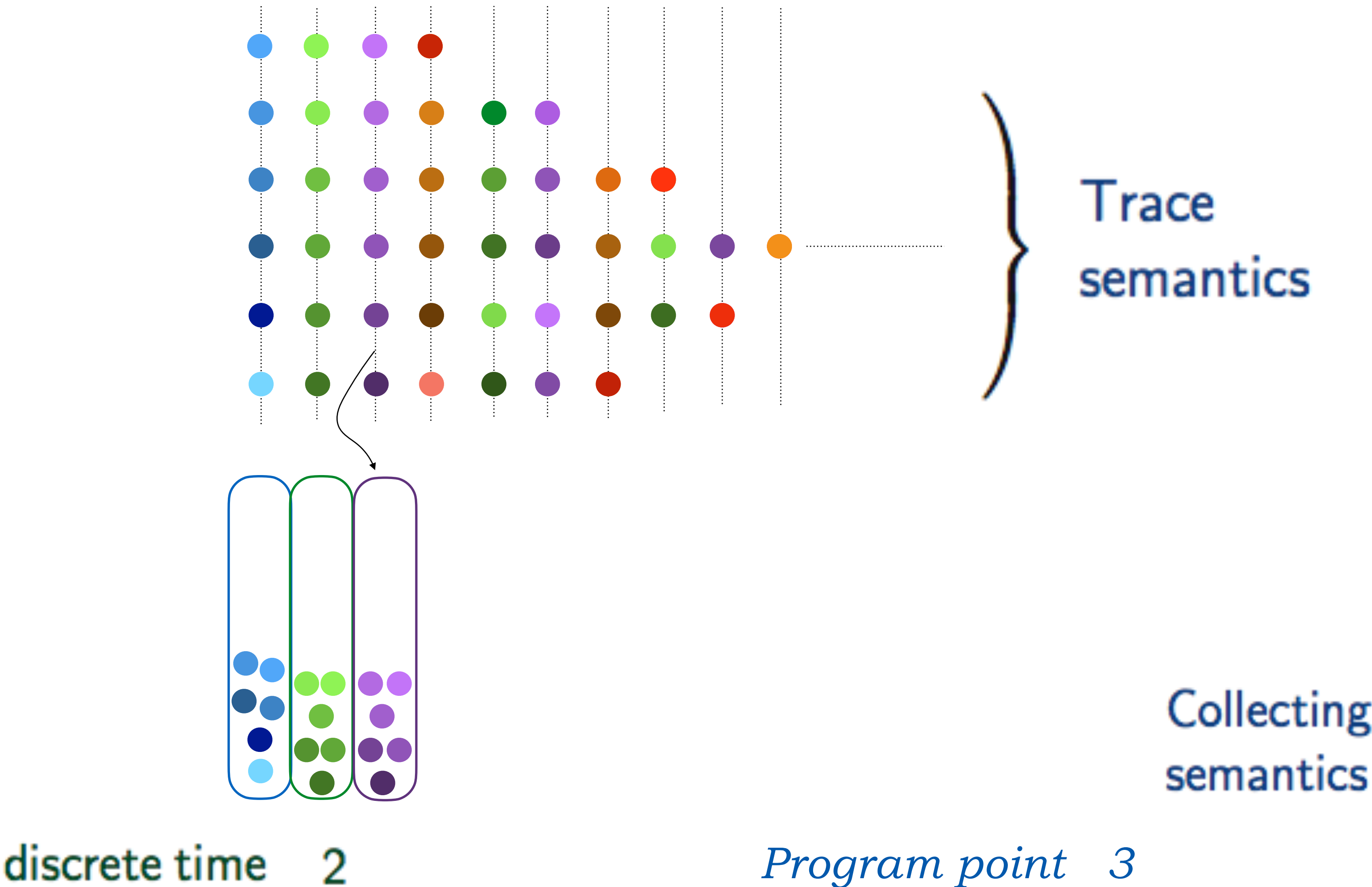
Collecting semantics



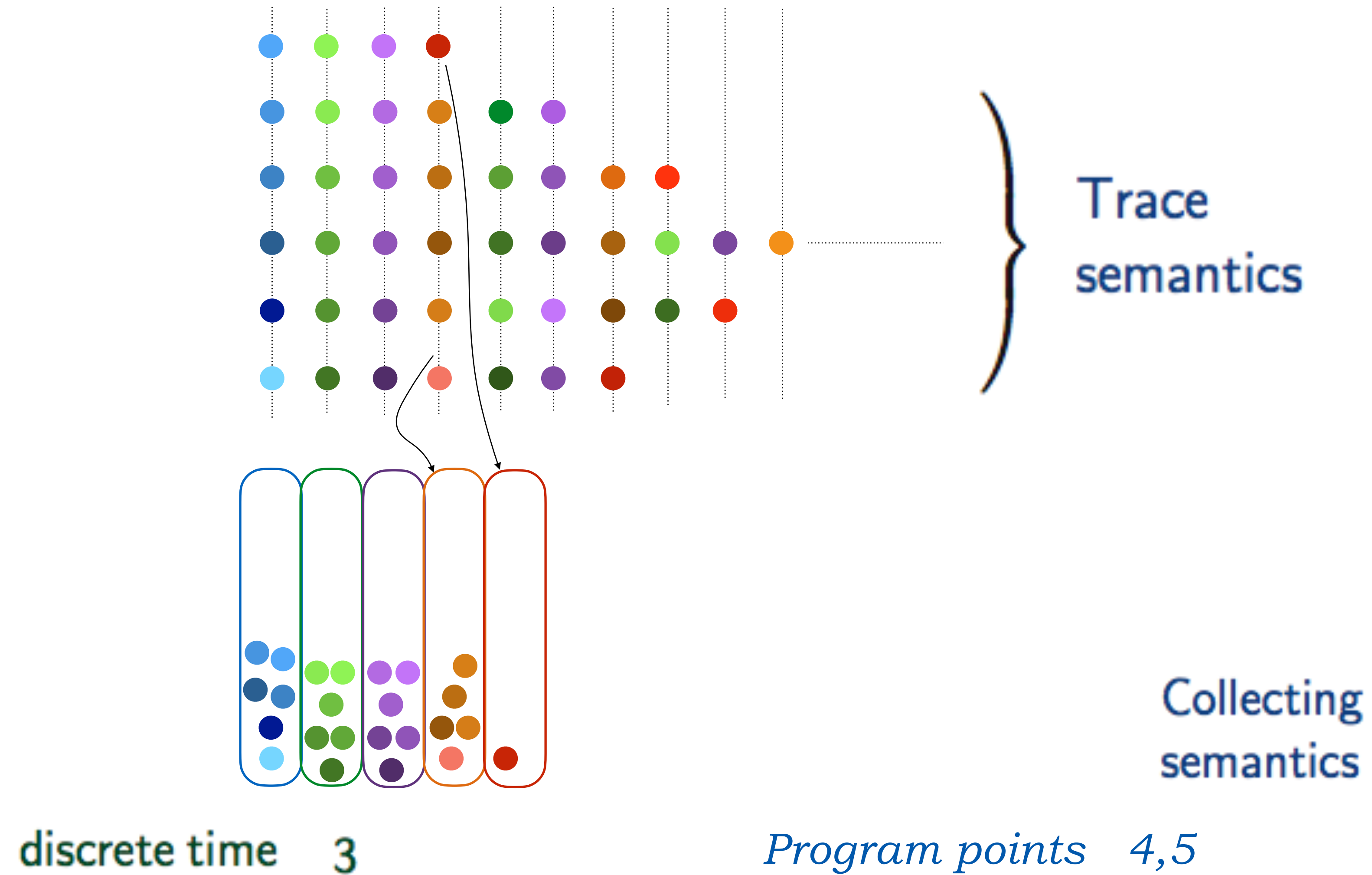
Collecting semantics



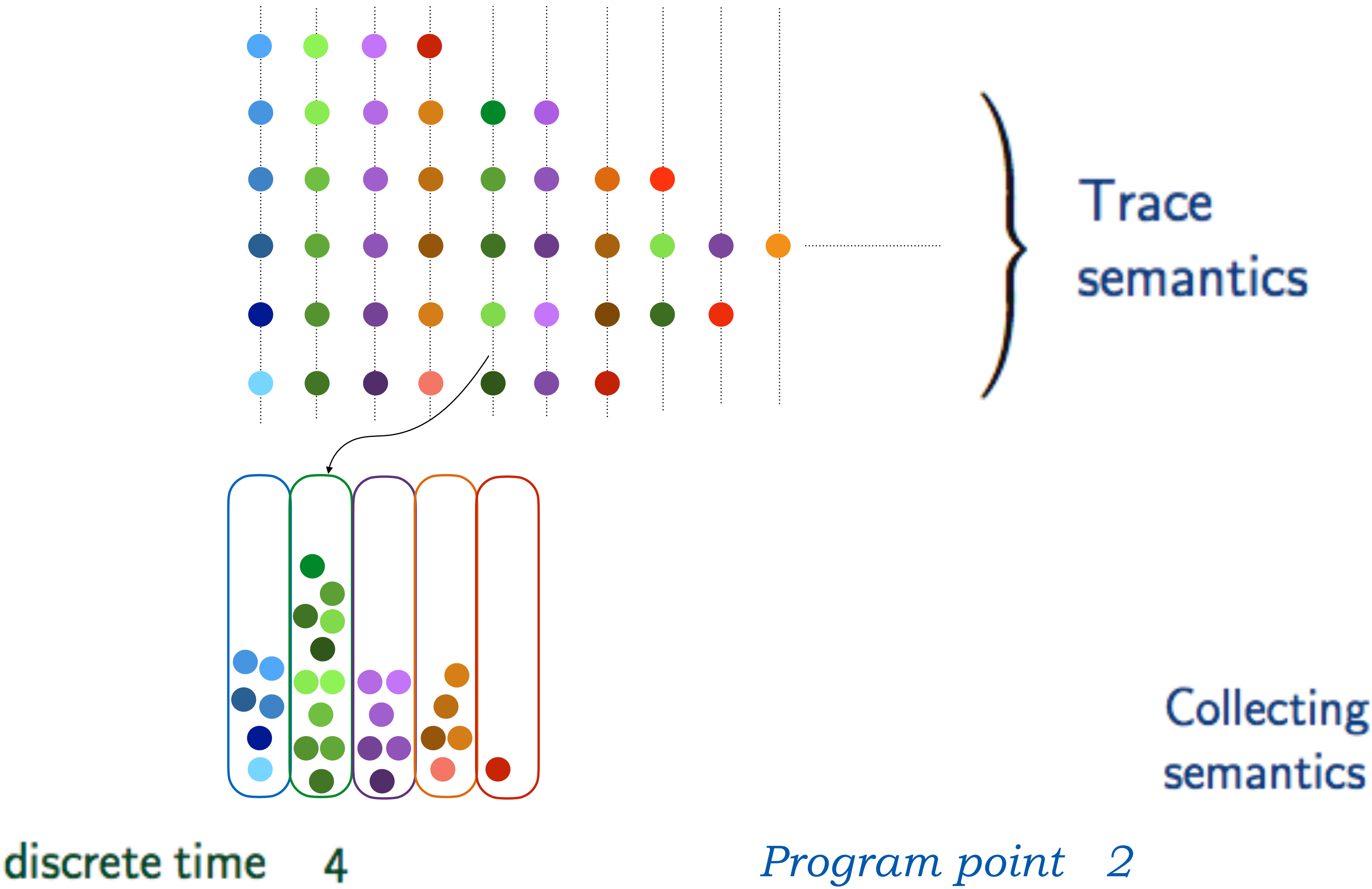
Collecting semantics



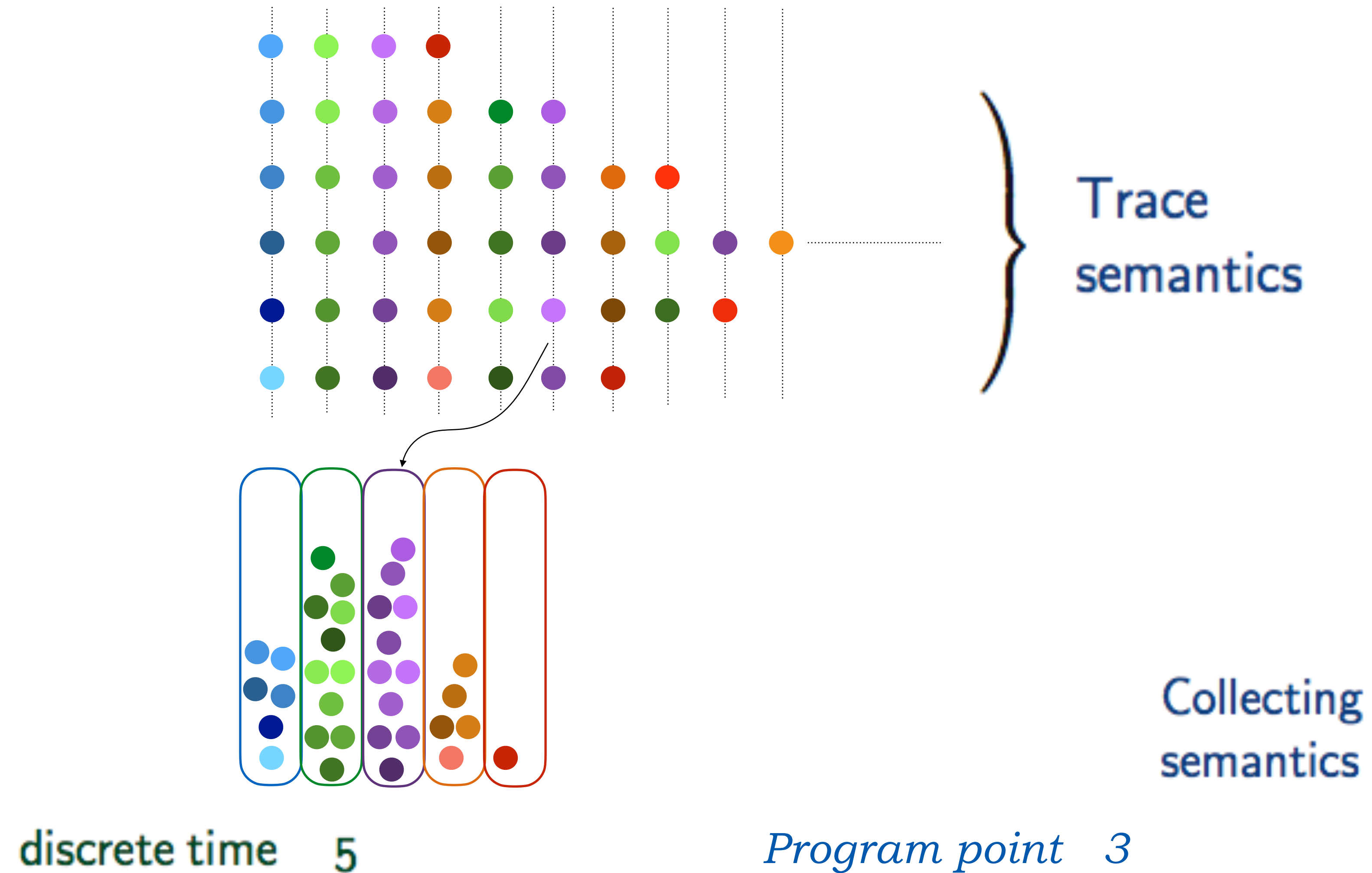
Collecting semantics



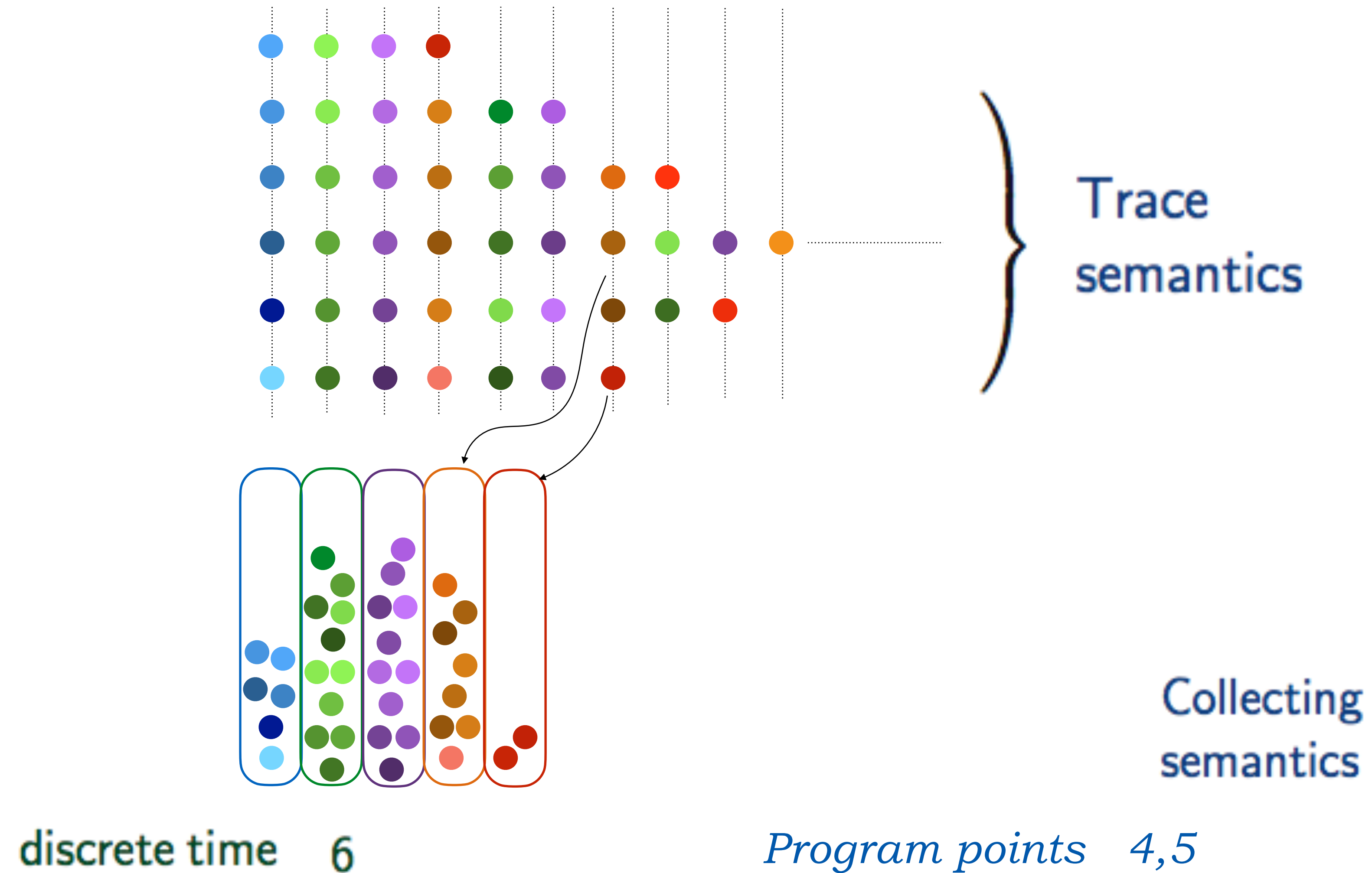
Collecting semantics



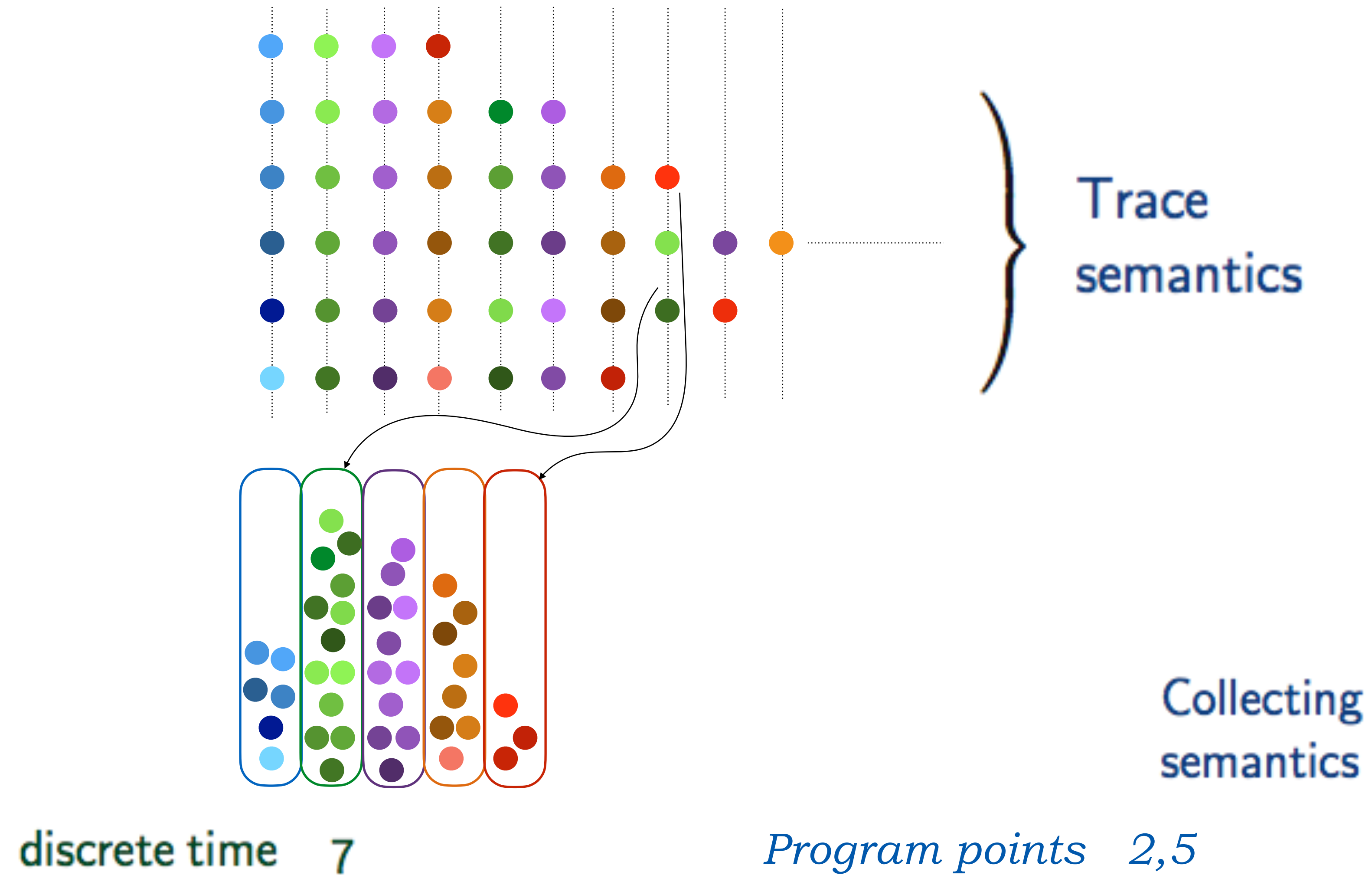
Collecting semantics



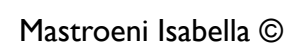
Collecting semantics



Collecting semantics

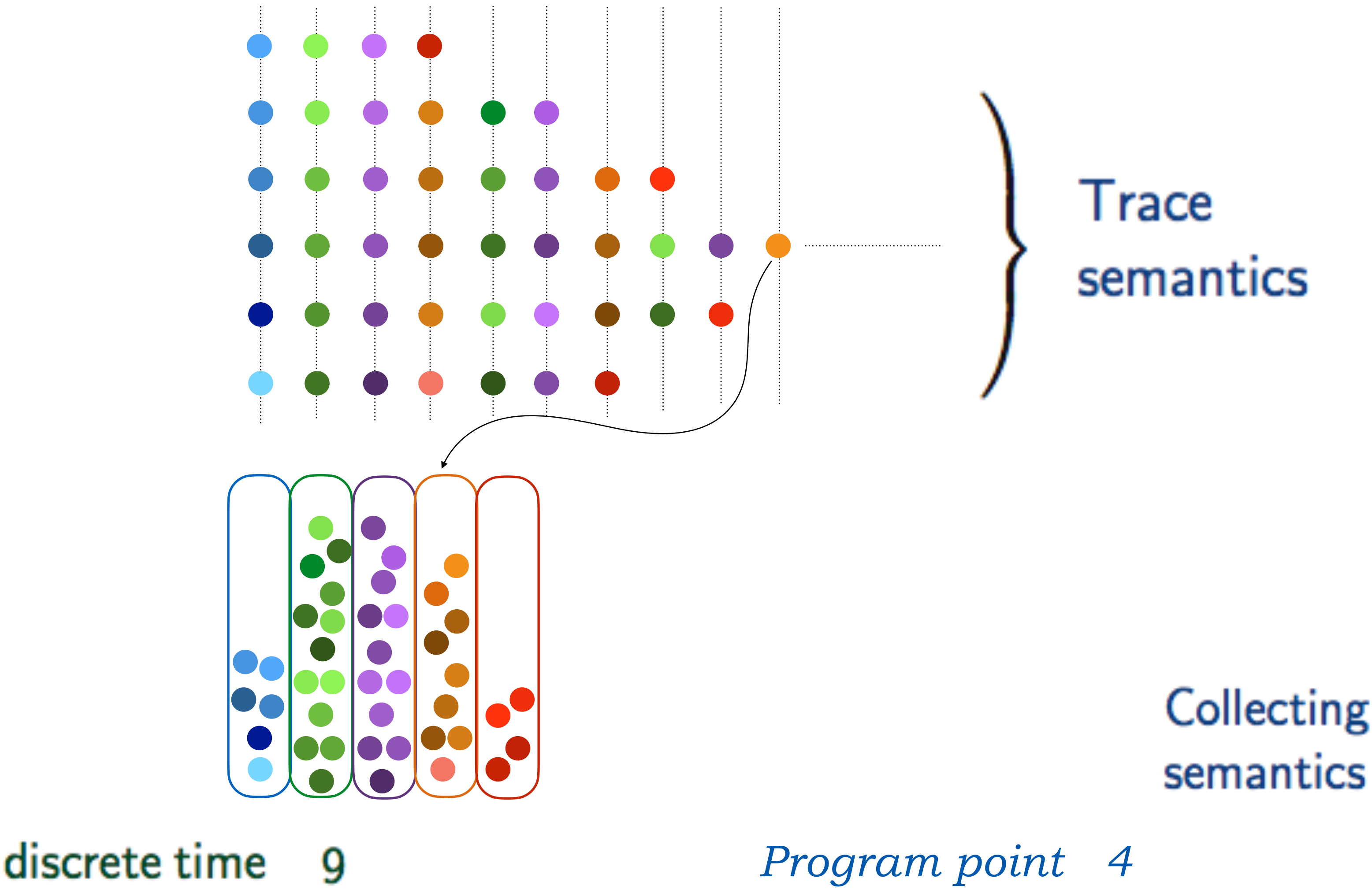


• • • • •



Collecting semantics

.....



What still we need

What still we need

- Collecting semantics: we collected computations, computing on program points rather than steps of computation

What still we need

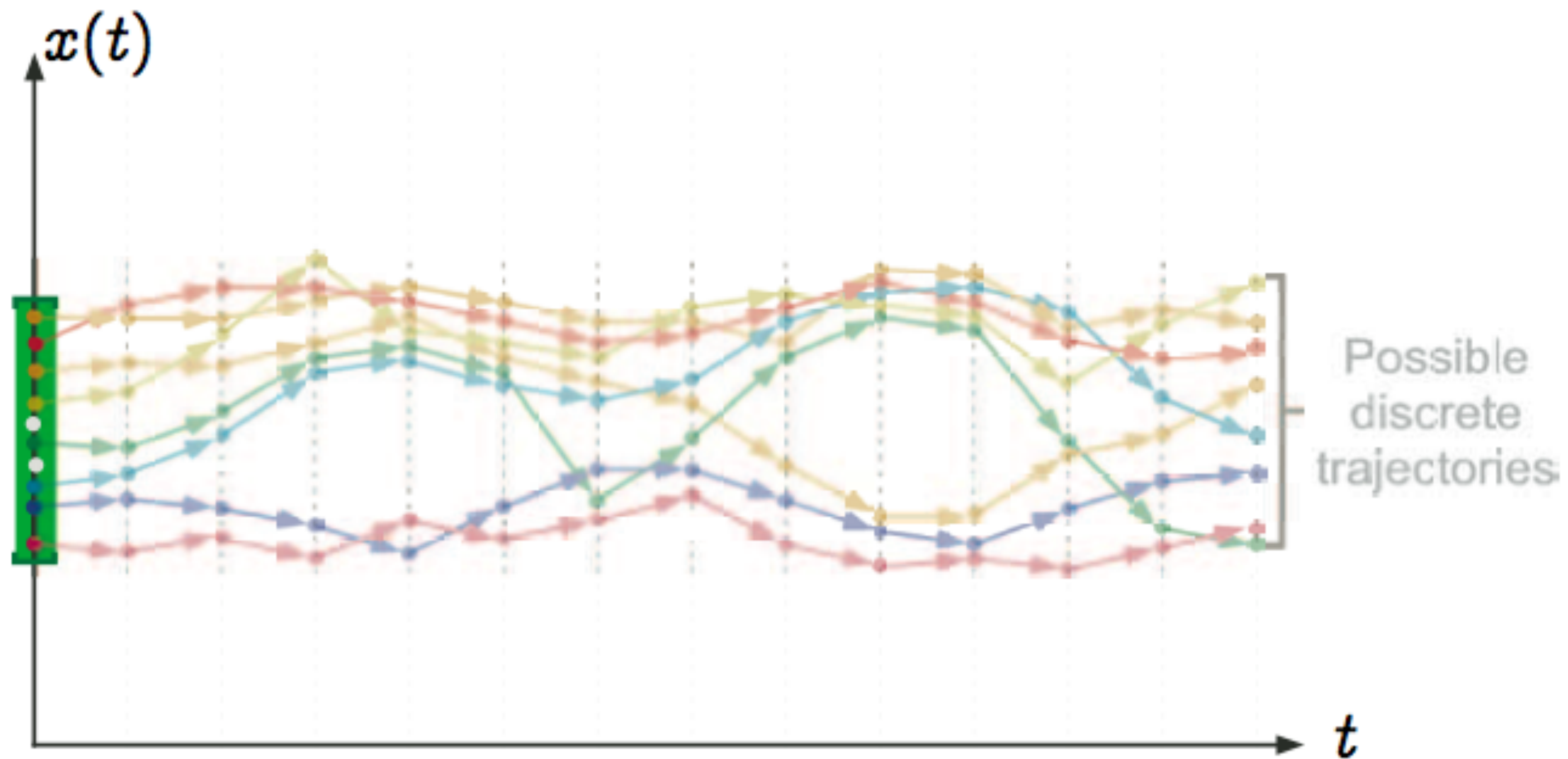
- Collecting semantics: we collected computations, computing on program points rather than steps of computation
- Still undecidable!

What still we need

- Collecting semantics: we collected computations, computing on program points rather than steps of computation
- Still undecidable!
- We have to further abstract computation, computing *properties* of reachable states at each program point

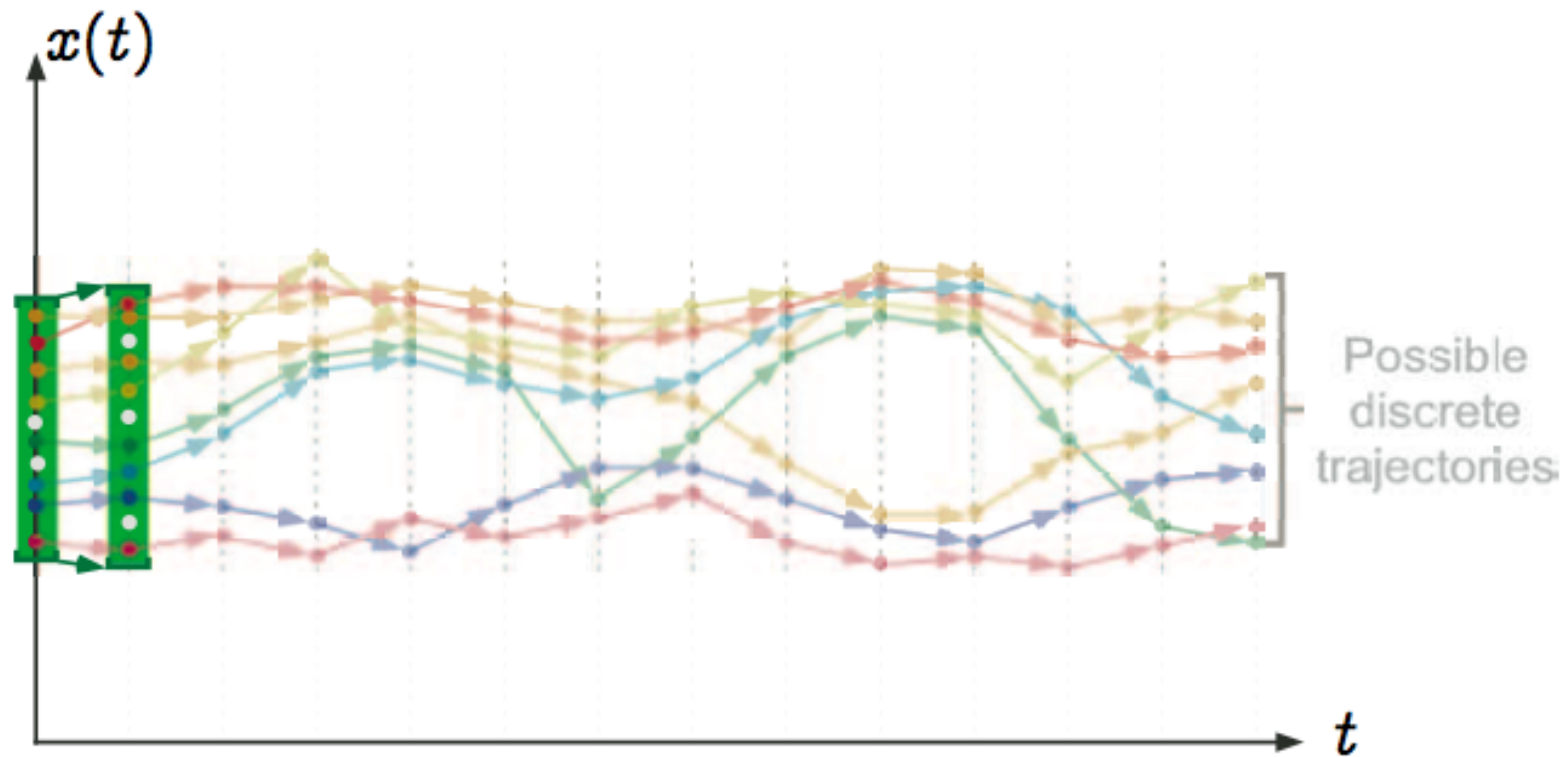
Computing on properties

.....

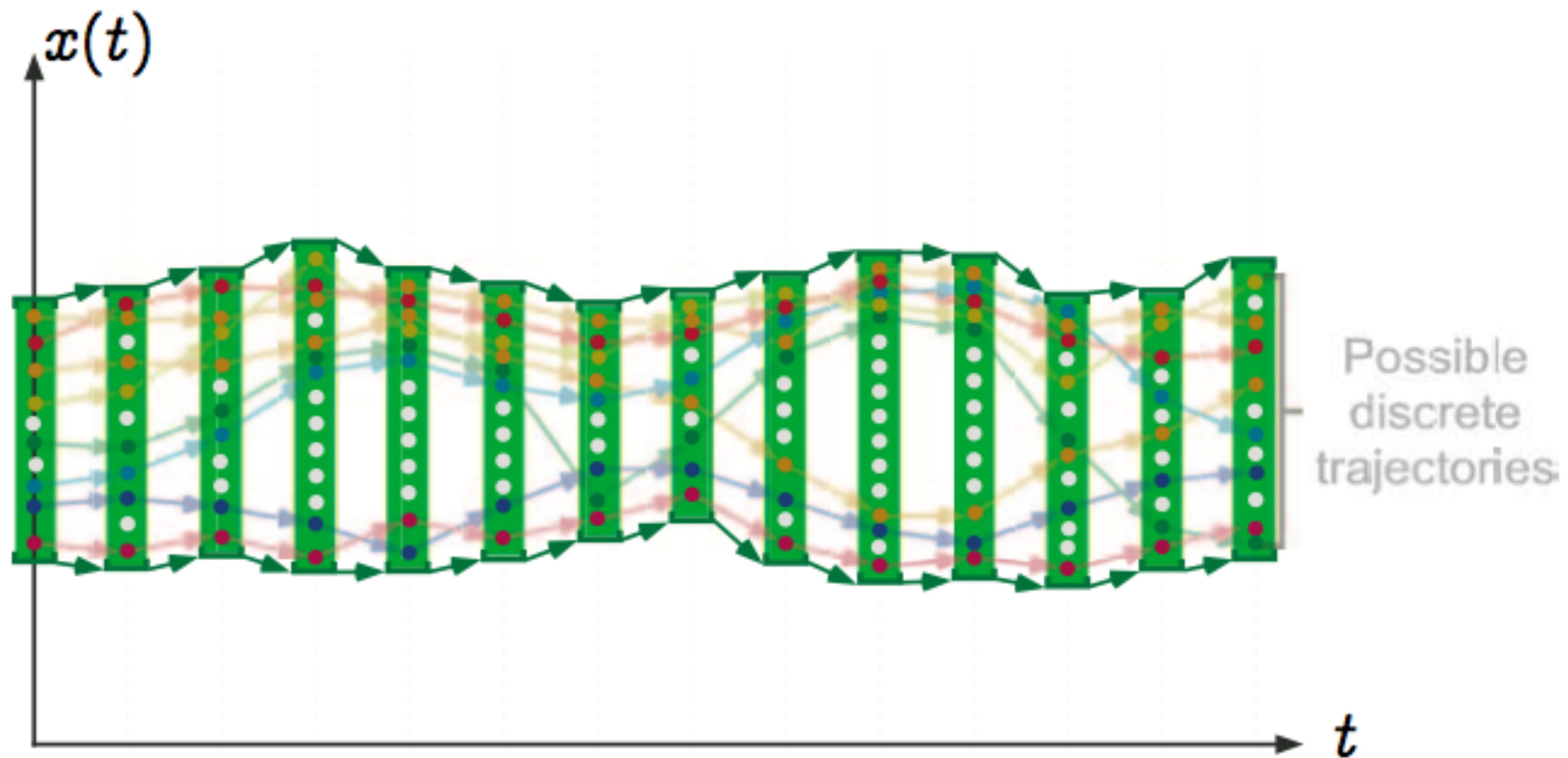


Computing on properties

.....



Computing on properties



What noise do we add?

What noise do we add?

- Collecting semantics: we have spurious executions due to collection (but only on reachable states)

What noise do we add?

- Collecting semantics: we have spurious executions due to collection (but only on reachable states)
- Collecting on properties: we add spurious states due to data approximation

Semantics as abstractions

.....

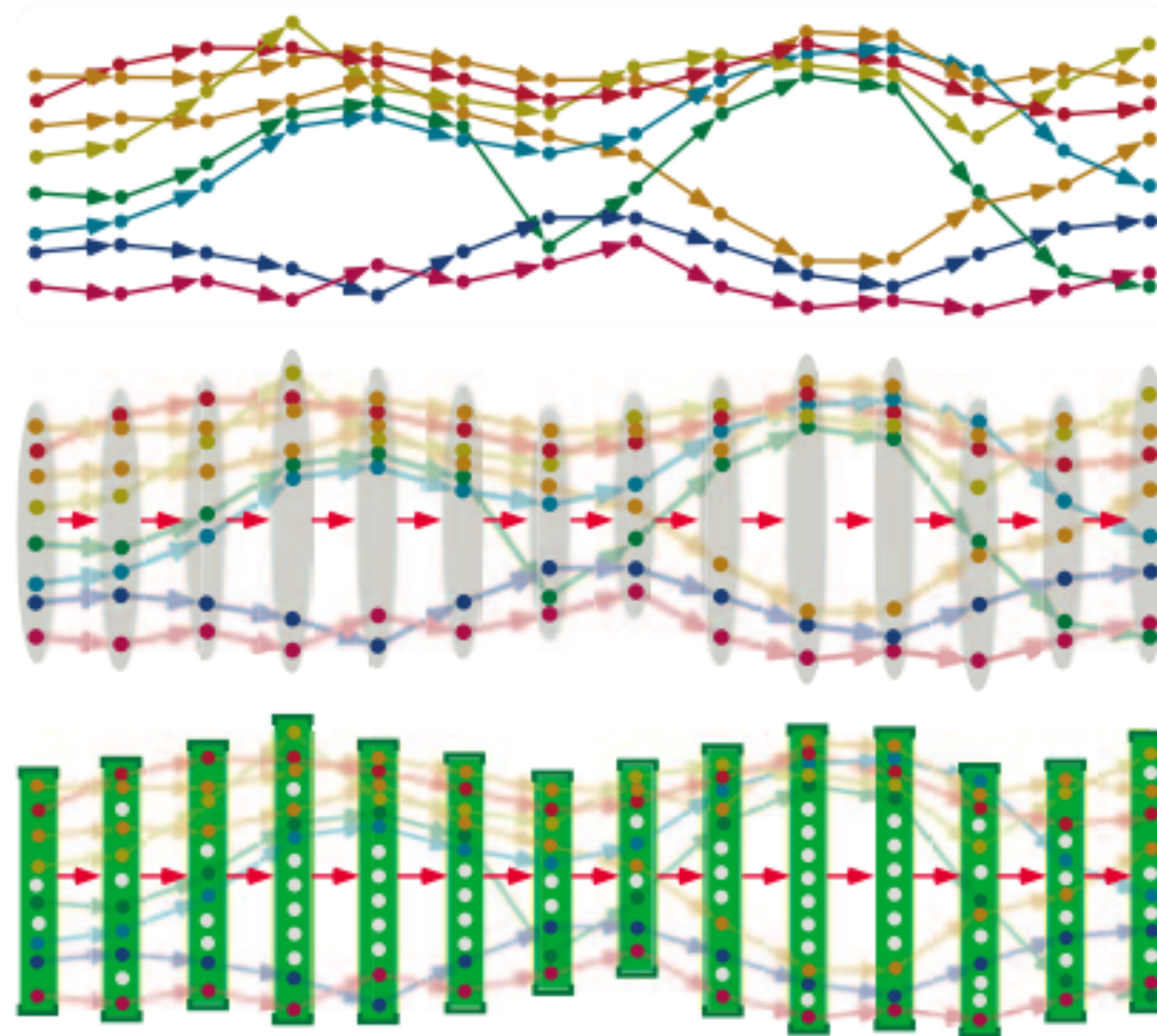
Set of traces:

$\alpha_1 \downarrow$

Trace of sets:

$\alpha_2 \downarrow$

Trace of intervals



Semantics as abstractions

.....

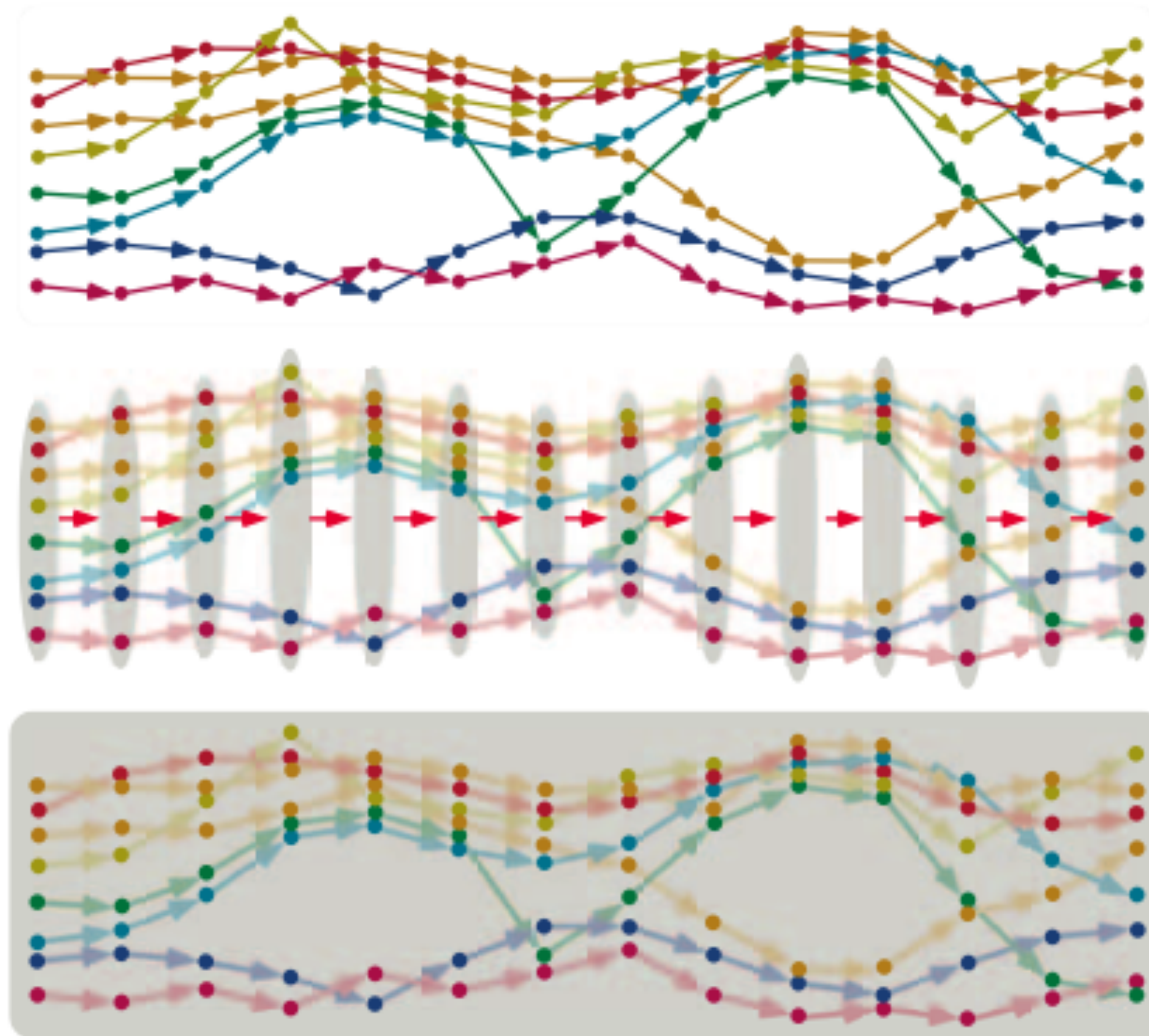
Set of traces:

$\alpha_1 \downarrow$

Trace of sets:

$\alpha_3 \downarrow$

Reachable states



Summarizing approximations

Summarizing approximations

- We consider properties (set) of states instead of concrete states

Summarizing approximations

- We consider properties (set) of states instead of concrete states
- We move the computation of sets (approximating trace semantics)

Summarizing approximations

- We consider properties (set) of states instead of concrete states
- We move the computation of sets (approximating trace semantics)
- We approximate computation computing collecting semantics, semantics collecting on program points

Summarizing approximations

- We consider properties (set) of states instead of concrete states
- We move the computation of sets (approximating trace semantics)
- We approximate computation computing collecting semantics, semantics collecting on program points
- Abstract computation can still be non terminating (and thus undecidable!)

Summarizing approximations

- We consider properties (set) of states instead of concrete states
- We move the computation of sets (approximating trace semantics)
- We approximate computation computing collecting semantics, semantics collecting on program points
- Abstract computation can still be non terminating (and thus undecidable!)
 - We will see how to fix this